

NeetCode 150 — Swift Field Guide

150 patterns · 18 families · annotated in Swift

This document is the complete print companion to the interactive Swift Field Guide website. All 150 problems appear here with idiomatic Swift solutions, plain-language approach notes, and complexity analysis. Nine flagship problems additionally include a written step-through of the algorithm in action — the interactive site animates those same nine frame by frame.

ROADMAP — ALL 18 FAMILIES

#	Family	Problems
01	Arrays & Hashing	9
02	Two Pointers	5
03	Sliding Window	6
04	Stack	7
05	Binary Search	7
06	Linked List	11
07	Trees	15
08	Tries	3
09	Heap / Priority Queue	7
10	Backtracking	9
11	Graphs	13
12	Advanced Graphs	6
13	1-D Dynamic Programming	12
14	2-D Dynamic Programming	11
15	Greedy	8
16	Intervals	6
17	Math & Geometry	8
18	Bit Manipulation	7

FAMILY 01

Arrays & Hashing

The workhorse family. Nearly every pattern here reduces to one idea: trade a linear scan for $O(1)$ lookups by remembering what you've already seen — in a Set, a Dictionary, or a fixed-size bucket array.

PLATE 01.01 Contains Duplicate [EASY]

Given an integer array `nums`, return true if any value appears at least twice.

[View on LeetCode ↗](#)

APPROACH — Set membership, $O(n)$ time / $O(n)$ space

```
func hasDuplicate(_ nums: [Int]) -> Bool {  
    var seen = Set<Int>()  
    for n in nums {  
        if seen.contains(n) { return true }  
        seen.insert(n)  
    }  
    return false  
}
```

TIME $O(n)$ SPACE $O(n)$

PLATE 01.02 Valid Anagram [EASY]

Given two strings `s` and `t`, return true if `t` is an anagram of `s`.

[View on LeetCode ↗](#)

APPROACH — Character frequency map, $O(n)$ time / $O(1)$ space (26 letters)

```
func isAnagram(_ s: String, _ t: String) -> Bool {  
    guard s.count == t.count else { return false }  
    var counts = [Character: Int]()  
    for c in s { counts[c, default: 0] += 1 }  
    for c in t {  
        counts[c, default: 0] -= 1  
        if counts[c]! < 0 { return false }  
    }  
    return true  
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 01.03 Two Sum [EASY]

Given an array of integers `nums` and a target, return indices of the two numbers that add up to target.

[View on LeetCode ↗](#)

APPROACH — One-pass hash map of value $\rightarrow$ index, $O(n)$ time / $O(n)$ space

```

func twoSum(_ nums: [Int], _ target: Int) -> [Int] {
    var seen = [Int: Int]() // value -> index

    for (i, n) in nums.enumerated() {
        let complement = target - n

        if let j = seen[complement] { return [j, i] }

        seen[n] = i
    }

    return []
}

```

TIME $O(n)$ SPACE $O(n)$

STEP-THROUGH — `nums = [2, 7, 11, 15]`, `target = 9` (animated on the companion site)

- `i=0`, `num=2`. Need complement 7. Map is empty — no match. Store `2 → 0`.
- `i=1`, `num=7`. Need complement 2. Map has `2 → 0`. Match found!
- Return `[0, 1]`, since `nums[0] + nums[1] = 2 + 7 = 9`.

PLATE 01.04 **Group Anagrams** [MEDIUM]

Given an array of strings, group the anagrams together.

[View on LeetCode ↗](#)

APPROACH — Sorted string as bucket key, $O(n \cdot k \log k)$ time

```

func groupAnagrams(_ strs: [String]) -> [[String]] {
    var groups = [String: [String]]()

    for s in strs {
        let key = String(s.sorted())

        groups[key, default: []].append(s)
    }

    return Array(groups.values)
}

```

TIME $O(n \cdot k \log k)$ SPACE $O(n \cdot k)$

PLATE 01.05 **Top K Frequent Elements** [MEDIUM]

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements.

[View on LeetCode ↗](#)

APPROACH — Bucket sort by frequency, $O(n)$ time (beats a heap's $O(n \log k)$)

```

func topKFrequent(_ nums: [Int], _ k: Int) -> [Int] {
    var counts = [Int: Int]()
    for n in nums { counts[n, default: 0] += 1 }

    var buckets = [[Int]](repeating: [], count: nums.count + 1)
    for (num, freq) in counts { buckets[freq].append(num) }

    var result = [Int]()
    for freq in stride(from: buckets.count - 1, through: 0, by: -1) {
        for num in buckets[freq] {
            result.append(num)
            if result.count == k { return result }
        }
    }
    return result
}

```

TIME **O(n)** SPACE **O(n)**

PLATE 01.06 **Product of Array Except Self** [MEDIUM]

Return an array where each element is the product of all other elements, without using division.

[View on LeetCode ↗](#)

APPROACH — Prefix pass then suffix pass, O(n) time / O(1) extra space

```

func productExceptSelf(_ nums: [Int]) -> [Int] {
    let n = nums.count

    var result = [Int](repeating: 1, count: n)

    var prefix = 1
    for i in 0..

```

TIME **O(n)** SPACE **O(1)** **extra**

STEP-THROUGH — `nums = [1, 2, 3, 4]` (animated on the companion site)

- Prefix pass: result becomes [1, 1, 2, 6] while prefix runs $1 \rightarrow 1 \rightarrow 2 \rightarrow 6 \rightarrow 24$.
- Suffix pass (right to left): multiply each `result[i]` by the running suffix product.
- $i=3: \times 1 \rightarrow 6$. $i=2: \times 4 \rightarrow 8$. $i=1: \times 12 \rightarrow 12$. $i=0: \times 24 \rightarrow 24$.
- Final result: [24, 12, 8, 6].

PLATE 01.07 **Valid Sudoku** [MEDIUM]

Determine if a 9x9 Sudoku board is valid — each row, column, and 3x3 sub-box contains no repeated digits.

[View on LeetCode ↗](#)

APPROACH — One Set per row, column, and box, O(1) since the board is fixed size

```

func isValidSudoku(_ board: [[Character]]) -> Bool {

    var rows = [Set<Character>](repeating: [], count: 9)

    var cols = [Set<Character>](repeating: [], count: 9)

    var boxes = [Set<Character>](repeating: [], count: 9)

    for r in 0..<9 {
        for c in 0..<9 {

            let val = board[r][c]

            if val == "." { continue }

            let b = (r / 3) * 3 + c / 3

            if rows[r].contains(val) || cols[c].contains(val) || boxes[b].contains(val) {

                return false

            }

            rows[r].insert(val)

            cols[c].insert(val)

            boxes[b].insert(val)

        }

    }

    return true

}

```

TIME **O(1)** SPACE **O(1)**

PLATE 01.08 **Encode and Decode Strings** [MEDIUM]

Design an algorithm to encode a list of strings to one string, and decode it back to the original list.

[View on LeetCode ↗](#)

APPROACH — Length-prefix encoding ("len#chars"), delimiter-safe for any characters

```

class Codec {

    func encode(_ strs: [String]) -> String {

        var result = ""

        for s in strs {

            result += "\(s.count)#\(s)"

        }

        return result

    }

    func decode(_ s: String) -> [String] {

        var result = [String]()

        var i = s.startIndex

        while i < s.endIndex {

            var j = i

            while s[j] != "#" { j = s.index(after: j) }

            let len = Int(s[i..

```

TIME **O(n)** SPACE **O(n)**

PLATE 01.09 **Longest Consecutive Sequence** [MEDIUM]

Given an unsorted array of integers, return the length of the longest run of consecutive integers, in O(n) time.

[View on LeetCode ↗](#)

APPROACH — Hash set + only start counting from sequence heads, O(n) time

```

func longestConsecutive(_ nums: [Int]) -> Int {

    let numSet = Set(nums)

    var longest = 0

    for n in numSet {

        // only start a count from the beginning of a sequence

        if !numSet.contains(n - 1) {

            var length = 1

            var current = n

            while numSet.contains(current + 1) {

                current += 1

                length += 1

            }

            longest = max(longest, length)

        }

    }

    return longest

}

```

TIME **$O(n)$** SPACE **$O(n)$**

STEP-THROUGH — `nums = [100, 4, 200, 1, 3, 2]` (animated on the companion site)

- Build a Set: {100, 4, 200, 1, 3, 2}.
- 100: 99 absent → sequence head. Walk forward: 101 absent. Length 1.
- 4: 3 present → not a head, skip (it will be counted from its head).
- 200: 199 absent → sequence head. Walk forward: 201 absent. Length 1.
- 1: 0 absent → sequence head! Walk: 2 present, 3 present, 4 present, 5 absent. Length 4.
- Longest = 4, from the chain 1 → 2 → 3 → 4.

FAMILY 02

Two Pointers

Two indices moving toward or away from each other collapse an $O(n^2)$ scan into $O(n)$, as long as the array has exploitable order — sorted, or with a monotonic property like height.

PLATE 02.01 Valid Palindrome [EASY]

Given a string, determine if it is a palindrome after converting to lowercase and removing non-alphanumeric characters.

[View on LeetCode ↗](#)

APPROACH — Two pointers converging from both ends, $O(n)$ time / $O(n)$ space

```
func isPalindrome(_ s: String) -> Bool {  
  
    let filtered = s.lowercased().filter { $0.isLetter || $0.isNumber }  
  
    let chars = Array(filtered)  
  
    var l = 0, r = chars.count - 1  
  
    while l < r {  
  
        if chars[l] != chars[r] { return false }  
  
        l += 1  
  
        r -= 1  
  
    }  
  
    return true  
  
}
```

TIME $O(n)$ SPACE $O(n)$

PLATE 02.02 Two Sum II — Input Array Is Sorted [MEDIUM]

Given a 1-indexed sorted array, return the indices of two numbers that add up to target.

[View on LeetCode ↗](#)

APPROACH — Two pointers exploit the sorted order directly, $O(n)$ time / $O(1)$ space

```
func twoSumII(_ numbers: [Int], _ target: Int) -> [Int] {  
  
    var l = 0, r = numbers.count - 1  
  
    while l < r {  
  
        let sum = numbers[l] + numbers[r]  
  
        if sum == target { return [l + 1, r + 1] }  
  
        else if sum < target { l += 1 }  
  
        else { r -= 1 }  
  
    }  
  
    return []  
  
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 02.03 3Sum [MEDIUM]

Given an integer array, return all unique triplets that sum to zero.

[View on LeetCode ↗](#)

APPROACH – Sort, fix one number, two-pointer the rest, $O(n^2)$ time

```
func threeSum(_ nums: [Int]) -> [[Int]] {  
    let nums = nums.sorted()  
    var result = [[Int]]()  
    for i in 0..  
nums.count {  
        if i > 0 && nums[i] == nums[i - 1] { continue }  
        if nums[i] > 0 { break }  
        var l = i + 1, r = nums.count - 1  
        while l < r {  
            let sum = nums[i] + nums[l] + nums[r]  
            if sum < 0 { l += 1 }  
            else if sum > 0 { r -= 1 }  
            else {  
                result.append([nums[i], nums[l], nums[r]])  
                l += 1  
                r -= 1  
                while l < r && nums[l] == nums[l - 1] { l += 1 }  
            }  
        }  
    }  
    return result  
}
```

TIME $O(n^2)$ SPACE $O(n)$

PLATE 02.04 Container With Most Water [MEDIUM]

Given heights of vertical lines, find two lines that together with the x-axis form the container holding the most water.

[View on LeetCode ↗](#)

APPROACH – Two pointers, always move the shorter wall inward, $O(n)$ time / $O(1)$ space

```

func maxArea(_ height: [Int]) -> Int {
    var l = 0, r = height.count - 1

    var best = 0

    while l < r {
        let area = min(height[l], height[r]) * (r - l)

        best = max(best, area)

        if height[l] < height[r] { l += 1 } else { r -= 1 }
    }

    return best
}

```

TIME **O(n)** SPACE **O(1)**

STEP-THROUGH — `height = [1,8,6,2,5,4,8,3,7]` (animated on the companion site)

- `l=0, r=8`. `area = min(1,7)×8 = 8`. `height[l] < height[r]`, so move `l → 1`.
- `l=1, r=8`. `area = min(8,7)×7 = 49` — new best. `height[r] < height[l]`, move `r → 7`.
- `l=1, r=7`. `area = min(8,3)×6 = 18`. Move `r → 6`.
- `l=1, r=6`. `area = min(8,8)×5 = 40`. Move `r → 5`.
- Pointers keep converging; nothing beats 49. Best area = 49.

PLATE 02.05 **Trapping Rain Water** [HARD]

Given elevation heights, compute how much rainwater can be trapped between the bars.

[View on LeetCode ↗](#)

APPROACH — Two pointers tracking running left/right maxima, **O(n)** time / **O(1)** space

```
func trap(_ height: [Int]) -> Int {  
    var l = 0, r = height.count - 1  
  
    var leftMax = 0, rightMax = 0  
  
    var water = 0  
  
    while l < r {  
        if height[l] < height[r] {  
            leftMax = max(leftMax, height[l])  
            water += leftMax - height[l]  
            l += 1  
        } else {  
            rightMax = max(rightMax, height[r])  
            water += rightMax - height[r]  
            r -= 1  
        }  
    }  
  
    return water  
}
```

TIME **$O(n)$** SPACE **$O(1)$**

FAMILY 03

Sliding Window

A window of indices $[l, r]$ expands and contracts over the array or string, keeping some invariant (a sum, a set of characters) valid without ever restarting the scan.

PLATE 03.01 Best Time to Buy and Sell Stock [EASY]

Given daily prices, find the maximum profit from one buy and one later sell.

[View on LeetCode ↗](#)

APPROACH – Track the running minimum price while scanning, $O(n)$ time / $O(1)$ space

```
func maxProfit(_ prices: [Int]) -> Int {  
    var minPrice = Int.max, maxProfit = 0  
    for price in prices {  
        minPrice = min(minPrice, price)  
        maxProfit = max(maxProfit, price - minPrice)  
    }  
    return maxProfit  
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 03.02 Longest Substring Without Repeating Characters [MEDIUM]

Given a string, find the length of the longest substring without repeating characters.

[View on LeetCode ↗](#)

APPROACH – Sliding window with a Set, shrink from the left on duplicates, $O(n)$ time

```
func lengthOfLongestSubstring(_ s: String) -> Int {  
    let chars = Array(s)  
    var window = Set<Character>()  
    var l = 0, best = 0  
    for r in 0..  
chars.count {  
        while window.contains(chars[r]) {  
            window.remove(chars[l])  
            l += 1  
        }  
        window.insert(chars[r])  
        best = max(best, r - l + 1)  
    }  
    return best  
}
```

TIME $O(n)$ SPACE $O(\min(n, \text{charset}))$

STEP-THROUGH — `s = "abba"` (animated on the companion site)

- `r=0 'a'`: window = {a}. Length 1.
- `r=1 'b'`: window = {a, b}. Length 2 — new best.
- `r=2 'b'`: duplicate! Shrink from the left until 'b' leaves the window, then insert. window = {b}.
- `r=3 'a'`: window = {b, a}. Length 2. Longest run without repeats = 2 ("ab" or "ba").

PLATE 03.03 Longest Repeating Character Replacement [MEDIUM]

Given a string and an integer `k`, find the length of the longest substring after replacing at most `k` characters so all characters are the same.

[View on LeetCode ↗](#)

APPROACH — Sliding window; shrink whenever (window size - most frequent char count) exceeds `k`

```
func characterReplacement(_ s: String, _ k: Int) -> Int {  
    let chars = Array(s)  
    var counts = [Character: Int]()  
    var l = 0, maxCount = 0, best = 0  
    for r in 0..  
chars.count {  
        counts[chars[r], default: 0] += 1  
        maxCount = max(maxCount, counts[chars[r]]!)  
        while (r - l + 1) - maxCount > k {  
            counts[chars[l]]! -= 1  
            l += 1  
        }  
        best = max(best, r - l + 1)  
    }  
    return best  
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 03.04 Permutation in String [MEDIUM]

Given strings `s1` and `s2`, return true if `s2` contains a permutation of `s1` as a substring.

[View on LeetCode ↗](#)

APPROACH — Fixed-size sliding window of character counts, compare against target counts

```

func checkInclusion(_ s1: String, _ s2: String) -> Bool {

    if s1.count > s2.count { return false }

    let a = Array(s1), b = Array(s2)

    var need = [Int](repeating: 0, count: 26)

    var window = [Int](repeating: 0, count: 26)

    let base = Character("a").asciiValue!

    for c in a { need[Int(c.asciiValue! - base)] += 1 }

    for i in 0...count {

        window[Int(b[i].asciiValue! - base)] += 1

        if i >= a.count {

            window[Int(b[i - a.count].asciiValue! - base)] -= 1

        }

        if i >= a.count - 1 && window == need { return true }

    }

    return false

}

```

TIME **O(n)** SPACE **O(1)**

PLATE 03.05 **Minimum Window Substring** [HARD]

Given strings s and t, find the smallest substring of s containing every character of t (with multiplicity).

[View on LeetCode ↗](#)

APPROACH — Sliding window with a "missing character" counter, O(n) time

```

func minWindow(_ s: String, _ t: String) -> String {

    if t.isEmpty || s.count < t.count { return "" }

    let sChars = Array(s)

    var need = [Character: Int]()

    for c in t { need[c, default: 0] += 1 }

    var missing = t.count

    var l = 0

    var bestLen = Int.max, bestStart = 0

    for r in 0..

```

TIME **O(n)** SPACE **O(1)**

PLATE 03.06 **Sliding Window Maximum** **[HARD]**

Given an array and window size k, return the maximum of each window as it slides across the array.

[View on LeetCode ↗](#)

APPROACH — Monotonic decreasing deque of indices, O(n) time


```
func maxSlidingWindow(_ nums: [Int], _ k: Int) -> [Int] {  
    var deque = [Int]() // indices, values decreasing  
    var result = [Int]()  
    for i in 0..  
nums.count {  
        while let last = deque.last, nums[last] < nums[i] {  
            deque.removeLast()  
        }  
        deque.append(i)  
        if deque.first! <= i - k { deque.removeFirst() }  
        if i >= k - 1 { result.append(nums[deque.first!]) }  
    }  
    return result  
}
```

TIME **O(n)** SPACE **O(k)**

FAMILY 04

Stack

Last-in-first-out order is the natural home for “find the nearest thing that matters” problems — nested brackets, monotonic sequences, and postfix expressions.

PLATE 04.01 Valid Parentheses [EASY]

Given a string of brackets, determine if every bracket is properly closed and nested.

[View on LeetCode ↗](#)

APPROACH — Stack of open brackets, match on close, $O(n)$ time / $O(n)$ space

```
func isValid(_ s: String) -> Bool {  
    var stack = [Character]()  
  
    let pairs: [Character: Character] = [")": "(", "]"": "[", "}": "{"]  
  
    for c in s {  
        if let open = pairs[c] {  
            if stack.popLast() != open { return false }  
        } else {  
            stack.append(c)  
        }  
    }  
  
    return stack.isEmpty  
}
```

TIME $O(n)$ SPACE $O(n)$

PLATE 04.02 Min Stack [MEDIUM]

Design a stack that supports push, pop, top, and retrieving the minimum element, all in $O(1)$.

[View on LeetCode ↗](#)

APPROACH — Each stack frame carries the running minimum alongside its value

```

class MinStack {

    private var stack = [(val: Int, min: Int)]()

    func push(_ val: Int) {

        let newMin = min(val, stack.last?.min ?? val)

        stack.append((val, newMin))

    }

    func pop() { stack.removeLast() }

    func top() -> Int { stack.last!.val }

    func getMin() -> Int { stack.last!.min }

}

```

TIME **O(1)** all ops SPACE **O(n)**

PLATE 04.03 **Evaluate Reverse Polish Notation** [MEDIUM]

Evaluate an arithmetic expression given in Reverse Polish (postfix) Notation.

[View on LeetCode ↗](#)

APPROACH – Push operands, pop two and apply on each operator, O(n) time

```

func evalRPN(_ tokens: [String]) -> Int {

    var stack = [Int]()

    let ops: Set<String> = ["+", "-", "*", "/"]

    for token in tokens {

        if ops.contains(token) {

            let b = stack.removeLast()

            let a = stack.removeLast()

            switch token {

                case "+": stack.append(a + b)

                case "-": stack.append(a - b)

                case "*": stack.append(a * b)

                default: stack.append(a / b)    // Swift's / truncates toward zero

            }

        } else {

            stack.append(Int(token)!)

        }

    }

    return stack.last!

}

```

TIME $O(n)$ SPACE $O(n)$

PLATE 04.04 **Generate Parentheses** [MEDIUM]

Given n pairs of parentheses, generate all combinations of well-formed parentheses.

[View on LeetCode ↗](#)

APPROACH — Backtracking; only add ')' when it wouldn't outnumber '('

```
func generateParenthesis(_ n: Int) -> [String] {  
    var result = [String]()  
    func backtrack(_ current: String, _ open: Int, _ close: Int) {  
        if current.count == 2 * n {  
            result.append(current)  
            return  
        }  
        if open < n { backtrack(current + "(", open + 1, close) }  
        if close < open { backtrack(current + ")", open, close + 1) }  
    }  
    backtrack("", 0, 0)  
    return result  
}
```

TIME $O(4^n / \sqrt{n})$ SPACE $O(n)$

PLATE 04.05 **Daily Temperatures** [MEDIUM]

Given daily temperatures, return how many days until a warmer temperature for each day (0 if none).

[View on LeetCode ↗](#)

APPROACH — Monotonic decreasing stack of unresolved indices, $O(n)$ time

```
func dailyTemperatures(_ temperatures: [Int]) -> [Int] {  
    var result = [Int](repeating: 0, count: temperatures.count)  
    var stack = [Int]() // indices, decreasing temperatures  
    for i in 0..        while let last = stack.last, temperatures[last] < temperatures[i] {  
            stack.removeLast()  
            result[last] = i - last  
        }  
        stack.append(i)  
    }  
    return result  
}
```

TIME $O(n)$ SPACE $O(n)$

STEP-THROUGH — `temperatures = [73,74,75,71,69,72]` (animated on the companion site)

- `i=0 (73)`: stack empty, push index 0.
- `i=1 (74)`: warmer than `temps[0]=73` — pop 0, `result[0] = 1`. Push index 1.
- `i=2 (75)`: warmer than `temps[1]=74` — pop 1, `result[1] = 1`. Push index 2.
- `i=3 (71)`, `i=4 (69)`: both cooler than 75 — just push. Stack grows to `[2,3,4]`.
- `i=5 (72)`: warmer than 69 and 71 — pop both, `result[4]=1`, `result[3]=2`. 75 stays on the stack.
- Final waits: `[1, 1, 0, 2, 1, 0]` — index 2 (75) never finds a warmer day.

PLATE 04.06 **Car Fleet** [MEDIUM]

Given car positions, speeds, and a target, count how many fleets arrive (cars that catch up merge into one fleet).

[View on LeetCode ↗](#)

APPROACH — Sort by position, treat arrival times as a monotonic stack of fleets

```
func carFleet(_ target: Int, _ position: [Int], _ speed: [Int]) -> Int {  
    let cars = zip(position, speed).sorted { $0.0 > $1.0 }  
    var fleets = 0  
    var lastTime = 0.0  
    for (pos, spd) in cars {  
        let time = Double(target - pos) / Double(spd)  
        if time > lastTime {  
            fleets += 1  
            lastTime = time  
        }  
    }  
    return fleets  
}
```

TIME $O(n \log n)$ SPACE $O(n)$

PLATE 04.07 **Largest Rectangle in Histogram** [HARD]

Given bar heights, find the area of the largest rectangle that fits within the histogram.

[View on LeetCode ↗](#)

APPROACH — Monotonic increasing stack of (index, height) pairs, $O(n)$ time

```
func largestRectangleArea(_ heights: [Int]) -> Int {  
    var stack = [(index: Int, height: Int)]()  
    var best = 0  
    for (i, h) in heights.enumerated() {  
        var start = i  
        while let top = stack.last, top.height > h {  
            best = max(best, top.height * (i - top.index))  
            start = top.index  
            stack.removeLast()  
        }  
        stack.append((start, h))  
    }  
    for (index, height) in stack {  
        best = max(best, height * (heights.count - index))  
    }  
    return best  
}
```

TIME **$O(n)$** SPACE **$O(n)$**

FAMILY 05

Binary Search

Not just for sorted arrays — any answer space with a monotonic yes/no property (can I finish in h hours? is the rotation point here?) can be binary searched.

PLATE 05.01 Binary Search [EASY]

Given a sorted array and a target, return its index, or -1 if not present.

[View on LeetCode ↗](#)

APPROACH — Classic binary search, $O(\log n)$ time / $O(1)$ space

```
func search(_ nums: [Int], _ target: Int) -> Int {  
    var lo = 0, hi = nums.count - 1  
    while lo <= hi {  
        let mid = lo + (hi - lo) / 2  
        if nums[mid] == target { return mid }  
        else if nums[mid] < target { lo = mid + 1 }  
        else { hi = mid - 1 }  
    }  
    return -1  
}
```

TIME $O(\log n)$ SPACE $O(1)$

PLATE 05.02 Search a 2D Matrix [MEDIUM]

Given a matrix sorted row-wise and column-wise (each row's first element greater than the previous row's last), find a target value.

[View on LeetCode ↗](#)

APPROACH — Treat the matrix as one flattened sorted array and binary search it

```
func searchMatrix(_ matrix: [[Int]], _ target: Int) -> Bool {  
    let rows = matrix.count, cols = matrix[0].count  
    var lo = 0, hi = rows * cols - 1  
    while lo <= hi {  
        let mid = lo + (hi - lo) / 2  
        let val = matrix[mid / cols][mid % cols]  
        if val == target { return true }  
        else if val < target { lo = mid + 1 }  
        else { hi = mid - 1 }  
    }  
    return false  
}
```

TIME $O(\log(m \cdot n))$ SPACE $O(1)$

PLATE 05.03 **Koko Eating Bananas** [MEDIUM]

Given piles of bananas and h hours, find the minimum eating speed to finish all piles in time.

[View on LeetCode ↗](#)

APPROACH – Binary search the answer space (eating speed) using a feasibility check

```
func minEatingSpeed(_ piles: [Int], _ h: Int) -> Int {
    var lo = 1, hi = piles.max()!
    while lo < hi {
        let mid = lo + (hi - lo) / 2
        let hoursNeeded = piles.reduce(0) { $0 + (($1 + mid - 1) / mid) }
        if hoursNeeded <= h { hi = mid } else { lo = mid + 1 }
    }
    return lo
}
```

TIME $O(n \log(\max(\text{piles})))$ SPACE $O(1)$

PLATE 05.04 **Find Minimum in Rotated Sorted Array** [MEDIUM]

Given a rotated sorted array with unique elements, find the minimum element in $O(\log n)$ time.

[View on LeetCode ↗](#)

APPROACH – Binary search comparing `nums[mid]` against `nums[hi]` to decide which half is rotated

```
func findMin(_ nums: [Int]) -> Int {
    var lo = 0, hi = nums.count - 1
    while lo < hi {
        let mid = lo + (hi - lo) / 2
        if nums[mid] > nums[hi] { lo = mid + 1 } else { hi = mid }
    }
    return nums[lo]
}
```

TIME $O(\log n)$ SPACE $O(1)$

STEP-THROUGH – `nums = [4,5,6,7,0,1,2]` (animated on the companion site)

- `lo=0, hi=6, mid=3` (value 7). `nums[mid] > nums[hi]` → the minimum is to the right. `lo = 4`.
- `lo=4, hi=6, mid=5` (value 1). `nums[mid] ≤ nums[hi]` → the minimum is at mid or left of it. `hi = 5`.
- `lo=4, hi=5, mid=4` (value 0). `nums[mid] ≤ nums[hi]` → `hi = 4`.
- `lo == hi == 4`. Minimum = `nums[4] = 0`.

PLATE 05.05 **Search in Rotated Sorted Array** [MEDIUM]

Given a rotated sorted array with unique elements and a target, return its index or -1.

[View on LeetCode ↗](#)

APPROACH – Binary search; at each step one half is guaranteed sorted, check if target lies in it


```

func searchRotated(_ nums: [Int], _ target: Int) -> Int {
    var lo = 0, hi = nums.count - 1

    while lo <= hi {
        let mid = lo + (hi - lo) / 2

        if nums[mid] == target { return mid }

        if nums[lo] <= nums[mid] {
            if nums[lo] <= target && target < nums[mid] { hi = mid - 1 }
            else { lo = mid + 1 }
        } else {
            if nums[mid] < target && target <= nums[hi] { lo = mid + 1 }
            else { hi = mid - 1 }
        }
    }

    return -1
}

```

TIME $O(\log n)$ SPACE $O(1)$

PLATE 05.06 Time Based Key-Value Store [MEDIUM]

Design a store where `set(key, value, timestamp)` records a value, and `get(key, timestamp)` returns the value set at the largest timestamp $\leq$ the given one.

[View on LeetCode ↗](#)

APPROACH — Append-only list per key, binary search for the right timestamp on get

```

class TimeMap {

    private var store = [String: [(timestamp: Int, value: String)]]()

    func set(_ key: String, _ value: String, _ timestamp: Int) {

        store[key, default: []].append((timestamp, value))

    }

    func get(_ key: String, _ timestamp: Int) -> String {

        guard let entries = store[key] else { return "" }

        var lo = 0, hi = entries.count - 1

        var result = ""

        while lo <= hi {

            let mid = lo + (hi - lo) / 2

            if entries[mid].timestamp <= timestamp {

                result = entries[mid].value

                lo = mid + 1

            } else {

                hi = mid - 1

            }

        }

        return result

    }

}

```

TIME $O(\log n)$ get, $O(1)$ set SPACE $O(n)$

PLATE 05.07 **Median of Two Sorted Arrays** [HARD]

Given two sorted arrays, find the median of the combined data in $O(\log(\min(m,n)))$ time.

[View on LeetCode ↗](#)

APPROACH — Binary search a valid partition point on the smaller array

```

func findMedianSortedArrays(_ nums1: [Int], _ nums2: [Int]) -> Double {

    let (a, b) = nums1.count <= nums2.count ? (nums1, nums2) : (nums2, nums1)

    let m = a.count, n = b.count

    var lo = 0, hi = m

    while lo <= hi {

        let i = lo + (hi - lo) / 2

        let j = (m + n + 1) / 2 - i

        let aLeft = i == 0 ? Int.min : a[i - 1]

        let aRight = i == m ? Int.max : a[i]

        let bLeft = j == 0 ? Int.min : b[j - 1]

        let bRight = j == n ? Int.max : b[j]

        if aLeft <= bRight && bLeft <= aRight {

            if (m + n) % 2 == 1 { return Double(max(aLeft, bLeft)) }

            return Double(max(aLeft, bLeft) + min(aRight, bRight)) / 2.0

        } else if aLeft > bRight {

            hi = i - 1

        } else {

            lo = i + 1

        }

    }

    return 0.0

}

```

TIME $O(\log(\min(m,n)))$ SPACE $O(1)$

FAMILY 06

Linked List

Pointer manipulation without an index. Fast/slow pointers, dummy heads, and careful reversal are the recurring tools.

PLATE 06.01 Reverse Linked List [EASY]

Given the head of a singly linked list, reverse it and return the new head.

[View on LeetCode ↗](#)

APPROACH – Iteratively relink each node's next pointer backward, $O(n)$ time / $O(1)$ space

```
class ListNode {
    var val: Int
    var next: ListNode?
    init(_ val: Int) { self.val = val }
}

func reverseList(_ head: ListNode?) -> ListNode? {
    var prev: ListNode? = nil
    var curr = head
    while curr != nil {
        let next = curr!.next
        curr!.next = prev
        prev = curr
        curr = next
    }
    return prev
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 06.02 Merge Two Sorted Lists [EASY]

Merge two sorted linked lists into one sorted list.

[View on LeetCode ↗](#)

APPROACH – Dummy head, splice the smaller node forward each step

```

func mergeTwoLists(_ list1: ListNode?, _ list2: ListNode?) -> ListNode? {

    let dummy = ListNode(0)

    var tail = dummy

    var a = list1, b = list2

    while let x = a, let y = b {

        if x.val <= y.val { tail.next = x; a = x.next } else { tail.next = y; b =
y.next }

        tail = tail.next!

    }

    tail.next = a ?? b

    return dummy.next

}

```

TIME $O(n + m)$ SPACE $O(1)$

PLATE 06.03 **Reorder List** [MEDIUM]

Reorder a list $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ into $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ in place.

[View on LeetCode ↗](#)

APPROACH — Find the middle (fast/slow), reverse the second half, merge the two halves

```

func reorderList(_ head: ListNode?) {
    guard head != nil else { return }
    var slow = head, fast = head
    while fast?.next != nil {
        slow = slow?.next
        fast = fast?.next?.next
    }
    var prev: ListNode? = nil
    var curr = slow?.next
    slow?.next = nil
    while curr != nil {
        let next = curr!.next
        curr!.next = prev
        prev = curr
        curr = next
    }
    var first = head, second = prev
    while second != nil {
        let n1 = first!.next
        let n2 = second!.next
        first!.next = second
        second!.next = n1
        first = n1
        second = n2
    }
}

```

TIME **O(n)** SPACE **O(1)**

PLATE 06.04 **Remove Nth Node From End of List** [MEDIUM]

Remove the nth node from the end of a linked list and return the head.

[View on LeetCode ↗](#)

APPROACH — Two pointers *n* apart; when the fast one hits the end, the slow one is at the target

```

func removeNthFromEnd(_ head: ListNode?, _ n: Int) -> ListNode? {

    let dummy = ListNode(0)

    dummy.next = head

    var fast: ListNode? = dummy, slow: ListNode? = dummy

    for _ in 0..

```

TIME **O(n)** SPACE **O(1)**

PLATE 06.05 **Copy List with Random Pointer** [MEDIUM]

Deep copy a linked list where each node has an extra pointer that can point to any node or nil.

[View on LeetCode ↗](#)

APPROACH — Map each original node to its clone, then wire next/random pointers in a second pass

```

class RandomNode {
    var val: Int
    var next: RandomNode?
    var random: RandomNode?
    init(_ val: Int) { self.val = val }
}

func copyRandomList(_ head: RandomNode?) -> RandomNode? {
    guard head != nil else { return nil }
    var map = [ObjectIdentifier: RandomNode]()
    var curr = head
    while let node = curr {
        map[ObjectIdentifier(node)] = RandomNode(node.val)
        curr = node.next
    }
    curr = head
    while let node = curr {
        let copy = map[ObjectIdentifier(node)]
        copy?.next = node.next.map { map[ObjectIdentifier($0)]! }
        copy?.random = node.random.map { map[ObjectIdentifier($0)]! }
        curr = node.next
    }
    return map[ObjectIdentifier(head!)]
}

```

TIME **O(n)** SPACE **O(n)**

PLATE 06.06 **Add Two Numbers** [MEDIUM]

Given two numbers as reversed-digit linked lists, add them and return the sum as a linked list.

[View on LeetCode ↗](#)

APPROACH — Walk both lists together, tracking carry, building the result as you go


```

func addTwoNumbers(_ l1: ListNode?, _ l2: ListNode?) -> ListNode? {

    let dummy = ListNode(0)

    var tail = dummy

    var a = l1, b = l2, carry = 0

    while a != nil || b != nil || carry != 0 {

        let sum = (a?.val ?? 0) + (b?.val ?? 0) + carry

        carry = sum / 10

        tail.next = ListNode(sum % 10)

        tail = tail.next!

        a = a?.next

        b = b?.next

    }

    return dummy.next

}

```

TIME $O(\max(n, m))$ SPACE $O(\max(n, m))$

PLATE 06.07 **Linked List Cycle** [EASY]

Determine if a linked list has a cycle.

[View on LeetCode ↗](#)

APPROACH — Floyd's fast/slow pointers; they meet if and only if a cycle exists

```

func hasCycle(_ head: ListNode?) -> Bool {

    var slow = head, fast = head

    while fast?.next != nil {

        slow = slow?.next

        fast = fast?.next?.next

        if slow === fast { return true }

    }

    return false

}

```

TIME $O(n)$ SPACE $O(1)$

PLATE 06.08 **Find the Duplicate Number** [MEDIUM]

Given an array of $n+1$ integers in $[1, n]$, find the one duplicate without modifying the array.

[View on LeetCode ↗](#)

APPROACH — Treat values as next-pointers; Floyd's cycle detection finds the duplicate as a cycle entry

```
func findDuplicate(_ nums: [Int]) -> Int {  
    var slow = nums[0], fast = nums[0]  
  
    repeat {  
        slow = nums[slow]  
        fast = nums[nums[fast]]  
    } while slow != fast  
  
    slow = nums[0]  
  
    while slow != fast {  
        slow = nums[slow]  
        fast = nums[fast]  
    }  
  
    return slow  
}
```

TIME **O(n)** SPACE **O(1)**

PLATE 06.09 **LRU Cache** **[MEDIUM]**

Design a Least Recently Used cache with O(1) get and put.

[View on LeetCode ↗](#)

APPROACH — Hash map of keys to nodes in a doubly linked list, ordered by recency

```

class LRUCache {

    private class Node {

        let key: Int

        var value: Int

        var prev: Node?

        var next: Node?

        init(_ key: Int, _ value: Int) { self.key = key; self.value = value }

    }

    private let capacity: Int

    private var map = [Int: Node]()

    private let head = Node(0, 0)

    private let tail = Node(0, 0)

    init(_ capacity: Int) {

        self.capacity = capacity

        head.next = tail

        tail.prev = head

    }

    private func remove(_ node: Node) {

        node.prev?.next = node.next

        node.next?.prev = node.prev

    }

    private func insertAtFront(_ node: Node) {

        node.next = head.next

        node.prev = head

        head.next?.prev = node

        head.next = node

    }

    func get(_ key: Int) -> Int {

        guard let node = map[key] else { return -1 }

        remove(node)

```

TIME $O(1)$ get/put SPACE $O(\text{capacity})$

PLATE 06.10 Merge k Sorted Lists [HARD]

Merge k sorted linked lists into one sorted list.

[View on LeetCode ↗](#)

APPROACH – Pairwise merge lists divide-and-conquer style, $O(n \log k)$ time

```
func mergeKLists(_ lists: [ListNode?]) -> ListNode? {  
    if lists.isEmpty { return nil }  
    var lists = lists  
    while lists.count > 1 {  
        var merged = [ListNode?]()  
        for i in stride(from: 0, to: lists.count, by: 2) {  
            let l1 = lists[i]  
            let l2 = i + 1 < lists.count ? lists[i + 1] : nil  
            merged.append(mergeTwoLists(l1, l2))  
        }  
        lists = merged  
    }  
    return lists[0]  
}
```

TIME $O(n \log k)$ SPACE $O(\log k)$

PLATE 06.11 Reverse Nodes in k-Group [HARD]

Given a linked list, reverse the nodes k at a time and return the modified list.

[View on LeetCode ↗](#)

APPROACH – Recursively check a full group exists, reverse it, then attach the recursively reversed remainder

```

func reverseKGroup(_ head: ListNode?, _ k: Int) -> ListNode? {

    var node = head

    var count = 0

    while node != nil && count < k {

        node = node!.next

        count += 1

    }

    if count < k { return head }

    var prev = reverseKGroup(node, k)

    var curr = head

    for _ in 0.. $k$  {

        let next = curr!.next

        curr!.next = prev

        prev = curr

        curr = next

    }

    return prev

}

```

TIME $O(n)$ SPACE $O(n/k)$ **recursion**

FAMILY 07

Trees

Recursion mirrors tree structure directly — most solutions are a base case plus “combine the answer from my children.”

PLATE 07.01 Invert Binary Tree [EASY]

Given the root of a binary tree, invert it (mirror left and right subtrees) and return the root.

[View on LeetCode ↗](#)

APPROACH — Recursively swap left and right children, $O(n)$ time

```
class TreeNode {  
    var val: Int  
    var left: TreeNode?  
    var right: TreeNode?  
    init(_ val: Int) { self.val = val }  
}  
  
func invertTree(_ root: TreeNode?) -> TreeNode? {  
    guard let root = root else { return nil }  
    let left = invertTree(root.left)  
    let right = invertTree(root.right)  
    root.left = right  
    root.right = left  
    return root  
}
```

TIME $O(n)$ SPACE $O(h)$

PLATE 07.02 Maximum Depth of Binary Tree [EASY]

Given the root of a binary tree, return its maximum depth.

[View on LeetCode ↗](#)

APPROACH — 1 + the deeper of the two child subtrees, $O(n)$ time

```
func maxDepth(_ root: TreeNode?) -> Int {  
    guard let root = root else { return 0 }  
    return 1 + max(maxDepth(root.left), maxDepth(root.right))  
}
```

TIME $O(n)$ SPACE $O(h)$

PLATE 07.03 Diameter of Binary Tree [EASY]

Return the length of the longest path between any two nodes in a binary tree.

[View on LeetCode ↗](#)

APPROACH – At every node, track the best left-depth + right-depth seen so far

```
func diameterOfBinaryTree(_ root: TreeNode?) -> Int {  
    var diameter = 0  
  
    func depth(_ node: TreeNode?) -> Int {  
        guard let node = node else { return 0 }  
  
        let left = depth(node.left)  
        let right = depth(node.right)  
  
        diameter = max(diameter, left + right)  
  
        return 1 + max(left, right)  
    }  
  
    depth(root)  
  
    return diameter  
}
```

TIME **O(n)** SPACE **O(h)**

PLATE 07.04 **Balanced Binary Tree** [EASY]

Determine if a binary tree is height-balanced (subtree heights never differ by more than 1).

[View on LeetCode ↗](#)

APPROACH – Bottom-up height check that short-circuits to -1 on imbalance

```
func isBalanced(_ root: TreeNode?) -> Bool {  
    func height(_ node: TreeNode?) -> Int {  
        guard let node = node else { return 0 }  
  
        let left = height(node.left)  
        if left == -1 { return -1 }  
  
        let right = height(node.right)  
        if right == -1 { return -1 }  
  
        if abs(left - right) > 1 { return -1 }  
  
        return 1 + max(left, right)  
    }  
  
    return height(root) != -1  
}
```

TIME **O(n)** SPACE **O(h)**

PLATE 07.05 **Same Tree** [EASY]

Given two binary trees, determine if they are structurally identical with the same values.

[View on LeetCode ↗](#)

APPROACH – Recursively compare values and both children

```

func isSameTree(_ p: TreeNode?, _ q: TreeNode?) -> Bool {

    if p == nil && q == nil { return true }

    guard let p = p, let q = q else { return false }

    return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right)

}

```

TIME $O(n)$ SPACE $O(h)$

PLATE 07.06 **Subtree of Another Tree** [EASY]

Given two trees root and subRoot, check whether subRoot appears as a subtree of root.

[View on LeetCode ↗](#)

APPROACH – At every node, try Same Tree; recurse otherwise

```

func isSubtree(_ root: TreeNode?, _ subRoot: TreeNode?) -> Bool {

    guard let root = root else { return subRoot == nil }

    if isSameTree(root, subRoot) { return true }

    return isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot)

}

```

TIME $O(n \cdot m)$ SPACE $O(h)$

PLATE 07.07 **Lowest Common Ancestor of a BST** [MEDIUM]

Given a binary search tree and two nodes p, q, find their lowest common ancestor.

[View on LeetCode ↗](#)

APPROACH – Use BST ordering: branch left or right, or stop when the values split

```

func lowestCommonAncestor(_ root: TreeNode?, _ p: TreeNode?, _ q: TreeNode?) -> TreeNode?
{
    var node = root

    while let curr = node {

        if p!.val < curr.val && q!.val < curr.val { node = curr.left }

        else if p!.val > curr.val && q!.val > curr.val { node = curr.right }

        else { return curr }

    }

    return nil
}

```

TIME $O(h)$ SPACE $O(1)$

PLATE 07.08 **Binary Tree Level Order Traversal** [MEDIUM]

Return the level-order traversal of a binary tree's node values (left to right, level by level).

[View on LeetCode ↗](#)

APPROACH – BFS, processing one full level of the queue per iteration


```

func levelOrder(_ root: TreeNode?) -> [[Int]] {
    guard let root = root else { return [] }
    var result = [[Int]]()
    var queue = [root]
    while !queue.isEmpty {
        var level = [Int]()
        var next = [TreeNode]()
        for node in queue {
            level.append(node.val)
            if let l = node.left { next.append(l) }
            if let r = node.right { next.append(r) }
        }
        result.append(level)
        queue = next
    }
    return result
}

```

TIME **O(n)** SPACE **O(n)**

PLATE 07.09 **Binary Tree Right Side View** [MEDIUM]

Return the values visible from the right side of a binary tree, top to bottom.

[View on LeetCode ↗](#)

APPROACH — BFS by level, keep the last node visited at each level

```

func rightSideView(_ root: TreeNode?) -> [Int] {

    guard let root = root else { return [] }

    var result = [Int]()

    var queue = [root]

    while !queue.isEmpty {

        result.append(queue.last!.val)

        var next = [TreeNode]()

        for node in queue {

            if let l = node.left { next.append(l) }

            if let r = node.right { next.append(r) }

        }

        queue = next

    }

    return result

}

```

TIME **O(n)** SPACE **O(n)**

PLATE 07.10 **Count Good Nodes in Binary Tree** [MEDIUM]

A node is “good” if no ancestor has a greater value. Count the good nodes in the tree.

[View on LeetCode ↗](#)

APPROACH – DFS carrying the max value seen on the path so far

```

func goodNodes(_ root: TreeNode?) -> Int {

    func dfs(_ node: TreeNode?, _ maxSoFar: Int) -> Int {

        guard let node = node else { return 0 }

        var count = node.val >= maxSoFar ? 1 : 0

        let newMax = max(maxSoFar, node.val)

        count += dfs(node.left, newMax)

        count += dfs(node.right, newMax)

        return count

    }

    return dfs(root, Int.min)

}

```

TIME **O(n)** SPACE **O(h)**

PLATE 07.11 **Validate Binary Search Tree** [MEDIUM]

Determine if a binary tree is a valid binary search tree.

[View on LeetCode ↗](#)

APPROACH – DFS carrying a valid (lower, upper) range for each node

```
func isValidBST(_ root: TreeNode?) -> Bool {
    func valid(_ node: TreeNode?, _ lower: Int, _ upper: Int) -> Bool {
        guard let node = node else { return true }
        if node.val <= lower || node.val >= upper { return false }
        return valid(node.left, lower, node.val) && valid(node.right, node.val, upper)
    }
    return valid(root, Int.min, Int.max)
}
```

TIME $O(n)$ SPACE $O(h)$

PLATE 07.12 **Kth Smallest Element in a BST** [MEDIUM]

Given a BST, return the kth smallest value in it.

[View on LeetCode ↗](#)

APPROACH – Iterative in-order traversal with an explicit stack, stop at the kth pop

```
func kthSmallest(_ root: TreeNode?, _ k: Int) -> Int {
    var stack = [TreeNode]()
    var node = root
    var count = k
    while node != nil || !stack.isEmpty {
        while let n = node {
            stack.append(n)
            node = n.left
        }
        node = stack.removeLast()
        count -= 1
        if count == 0 { return node!.val }
        node = node!.right
    }
    return -1
}
```

TIME $O(h + k)$ SPACE $O(h)$

PLATE 07.13 **Construct Binary Tree from Preorder and Inorder Traversal** [MEDIUM]

Given preorder and inorder traversal arrays, reconstruct and return the binary tree.

[View on LeetCode ↗](#)

APPROACH – Preorder gives roots in order; inorder index tells you the split between left/right subtrees

```

func buildTree(_ preorder: [Int], _ inorder: [Int]) -> TreeNode? {

    var preIndex = 0

    var inorderIndexMap = [Int: Int]()

    for (i, v) in inorder.enumerated() { inorderIndexMap[v] = i }

    func build(_ left: Int, _ right: Int) -> TreeNode? {

        guard left <= right else { return nil }

        let rootVal = preorder[preIndex]

        preIndex += 1

        let root = TreeNode(rootVal)

        let mid = inorderIndexMap[rootVal]!

        root.left = build(left, mid - 1)

        root.right = build(mid + 1, right)

        return root

    }

    return build(0, inorder.count - 1)

}

```

TIME **O(n)** SPACE **O(n)**

PLATE 07.14 **Binary Tree Maximum Path Sum** [HARD]

Find the maximum path sum between any two nodes in a binary tree (path need not pass through the root).

[View on LeetCode ↗](#)

APPROACH – DFS returning the best downward gain from each node, tracking the best full path globally

```

func maxPathSum(_ root: TreeNode?) -> Int {

    var best = Int.min

    func gain(_ node: TreeNode?) -> Int {

        guard let node = node else { return 0 }

        let left = max(gain(node.left), 0)

        let right = max(gain(node.right), 0)

        best = max(best, node.val + left + right)

        return node.val + max(left, right)

    }

    gain(root)

    return best

}

```

TIME $O(n)$ SPACE $O(h)$

PLATE 07.15 Serialize and Deserialize Binary Tree [HARD]

Design an algorithm to serialize a binary tree to a string and deserialize it back.

[View on LeetCode ↗](#)

APPROACH — Preorder DFS with “#” markers for nil children

```
class TreeCodec {

    func serialize(_ root: TreeNode?) -> String {

        var result = [String]()

        func dfs(_ node: TreeNode?) {

            guard let node = node else { result.append("#"); return }

            result.append(String(node.val))

            dfs(node.left)

            dfs(node.right)

        }

        dfs(root)

        return result.joined(separator: ",")

    }

    func deserialize(_ data: String) -> TreeNode? {

        let values = data.split(separator: ",").map(String.init)

        var index = 0

        func dfs() -> TreeNode? {

            let val = values[index]

            index += 1

            if val == "#" { return nil }

            let node = TreeNode(Int(val)!)

            node.left = dfs()

            node.right = dfs()

            return node

        }

        return dfs()

    }

}
```

TIME $O(n)$ SPACE $O(n)$

FAMILY 08

Tries

A tree shaped by shared prefixes. Built once, it turns prefix and wildcard lookups into a walk of depth equal to the word length.

PLATE 08.01 **Implement Trie (Prefix Tree)** [MEDIUM]

Implement a Trie with insert, search, and startsWith operations.

[View on LeetCode ↗](#)

APPROACH — A tree of characters; each node holds a dictionary of children and an end-of-word flag

```

class TrieNode {

    var children = [Character: TrieNode]()

    var isEnd = false
}

class Trie {

    private let root = TrieNode()

    func insert(_ word: String) {

        var node = root

        for c in word {

            if node.children[c] == nil { node.children[c] = TrieNode() }

            node = node.children[c]!

        }

        node.isEnd = true

    }

    func search(_ word: String) -> Bool {

        guard let node = find(word) else { return false }

        return node.isEnd

    }

    func startsWith(_ prefix: String) -> Bool {

        find(prefix) != nil

    }

    private func find(_ word: String) -> TrieNode? {

        var node = root

        for c in word {

            guard let next = node.children[c] else { return nil }

            node = next

        }

        return node

    }

}

```

TIME $O(k)$ per op SPACE $O(\text{total characters})$

PLATE 08.02 Design Add and Search Words Data Structure [MEDIUM]

Design a data structure supporting addWord and search, where search may include '.' as a wildcard for any letter.

[View on LeetCode ↗](#)

APPROACH – Trie plus DFS at search time to branch over all children on a wildcard

```
class WordDictionary {
    private let root = TrieNode()

    func addWord(_ word: String) {
        var node = root
        for c in word {
            if node.children[c] == nil { node.children[c] = TrieNode() }
            node = node.children[c]!
        }
        node.isEnd = true
    }

    func search(_ word: String) -> Bool {
        let chars = Array(word)
        func dfs(_ node: TrieNode, _ i: Int) -> Bool {
            if i == chars.count { return node.isEnd }
            let c = chars[i]
            if c == "." {
                for child in node.children.values {
                    if dfs(child, i + 1) { return true }
                }
                return false
            }
            guard let next = node.children[c] else { return false }
            return dfs(next, i + 1)
        }
        return dfs(root, 0)
    }
}
```

TIME $O(k)$ average, $O(26^k)$ worst case SPACE $O(\text{total characters})$

PLATE 08.03 **Word Search II** **[HARD]**

Given a board of characters and a list of words, return all words that can be formed by sequentially adjacent cells.

[View on LeetCode ↗](#)

APPROACH — Build a trie of all target words, then DFS the board once, pruning branches the trie rules out

```

func findWords(_ board: [[Character]], _ words: [String]) -> [String] {

    let root = TrieNode()

    for w in words {

        var node = root

        for c in w {

            if node.children[c] == nil { node.children[c] = TrieNode() }

            node = node.children[c]!

        }

        node.isEnd = true

    }

    var board = board

    let rows = board.count, cols = board[0].count

    var found = Set<String>()

    func dfs(_ r: Int, _ c: Int, _ node: TrieNode, _ path: String) {

        if r < 0 || r >= rows || c < 0 || c >= cols || board[r][c] == "#" { return }

        let ch = board[r][c]

        guard let next = node.children[ch] else { return }

        let newPath = path + String(ch)

        if next.isEnd { found.insert(newPath) }

        board[r][c] = "#"

        dfs(r + 1, c, next, newPath)

        dfs(r - 1, c, next, newPath)

        dfs(r, c + 1, next, newPath)

        dfs(r, c - 1, next, newPath)

        board[r][c] = ch

    }

    for r in 0..

```

TIME $O(\text{rows} \cdot \text{cols} \cdot 4^L)$ SPACE $O(\text{total characters in trie})$

FAMILY 09

Heap / Priority Queue

Whenever you need the running min, max, or top-k without re-sorting on every insert, a heap keeps that answer at the root in $O(\log n)$. Swift has no built-in heap, so the family opens with one reusable implementation.

PLATE 09.00 **Reusable Heap (used by every problem below)** [EASY]

Swift has no built-in heap, so every problem in this family shares one generic binary-heap implementation, parameterized by a comparator.

APPROACH — Array-backed binary heap with sift-up / sift-down, $O(\log n)$ push and pop

```

struct Heap<T> {

    private var data = [T]()

    private let areInIncreasingOrder: (T, T) -> Bool

    init(sort: @escaping (T, T) -> Bool) { self.areInIncreasingOrder = sort }

    var isEmpty: Bool { data.isEmpty }

    var count: Int { data.count }

    var peek: T? { data.first }

    mutating func push(_ value: T) {

        data.append(value)

        siftUp(data.count - 1)

    }

    mutating func pop() -> T? {

        guard !data.isEmpty else { return nil }

        data.swapAt(0, data.count - 1)

        let value = data.removeLast()

        siftDown(0)

        return value

    }

    private mutating func siftUp(_ index: Int) {

        var child = index

        var parent = (child - 1) / 2

        while child > 0 && areInIncreasingOrder(data[child], data[parent]) {

            data.swapAt(child, parent)

            child = parent

            parent = (child - 1) / 2

        }

    }

    private mutating func siftDown(_ index: Int) {

        var parent = index

```

TIME $O(\log n)$ push/pop SPACE $O(n)$

PLATE 09.01 **Kth Largest Element in a Stream** [EASY]

Design a class that keeps track of the kth largest element as new numbers stream in.

[View on LeetCode ↗](#)

APPROACH – A min-heap capped at size k; its root is always the kth largest

```
class KthLargest {  
    private var heap = Heap<Int>(sort: <)  
    private let k: Int  
  
    init(_ k: Int, _ nums: [Int]) {  
        self.k = k  
        for n in nums { _ = add(n) }  
    }  
  
    func add(_ val: Int) -> Int {  
        heap.push(val)  
        if heap.count > k { _ = heap.pop() }  
        return heap.peek!  
    }  
}
```

TIME $O(\log k)$ per add SPACE $O(k)$

PLATE 09.02 **Last Stone Weight** [EASY]

Repeatedly smash the two heaviest stones together until at most one remains; return its weight.

[View on LeetCode ↗](#)

APPROACH – Max-heap; pop the two heaviest, push back the difference

```
func lastStoneWeight(_ stones: [Int]) -> Int {  
    var heap = Heap<Int>(sort: >)  
    for s in stones { heap.push(s) }  
    while heap.count > 1 {  
        let a = heap.pop()!  
        let b = heap.pop()!  
        if a != b { heap.push(a - b) }  
    }  
    return heap.peek ?? 0  
}
```

TIME $O(n \log n)$ SPACE $O(n)$

PLATE 09.03 K Closest Points to Origin [MEDIUM]

Given points on a plane, return the k points closest to the origin.

[View on LeetCode ↗](#)

APPROACH – Max-heap of size k by squared distance; anything farther than the root gets discarded

```
func kClosest(_ points: [[Int]], _ k: Int) -> [[Int]] {  
    var heap = Heap<Int>(sort: { $0[0]*$0[0] + $0[1]*$0[1] > $1[0]*$1[0] +  
    $1[1]*$1[1] })  
  
    for p in points {  
        heap.push(p)  
  
        if heap.count > k { _ = heap.pop() }  
    }  
  
    var result = [[Int]]()  
  
    while let p = heap.pop() { result.append(p) }  
  
    return result  
}
```

TIME $O(n \log k)$ SPACE $O(k)$

PLATE 09.04 Kth Largest Element in an Array [MEDIUM]

Find the kth largest element in an unsorted array.

[View on LeetCode ↗](#)

APPROACH – Min-heap capped at size k; the root ends up being the kth largest

```
func findKthLargest(_ nums: [Int], _ k: Int) -> Int {  
    var heap = Heap<Int>(sort: <)  
  
    for n in nums {  
        heap.push(n)  
  
        if heap.count > k { _ = heap.pop() }  
    }  
  
    return heap.peek!  
}
```

TIME $O(n \log k)$ SPACE $O(k)$

PLATE 09.05 Task Scheduler [MEDIUM]

Given CPU tasks and a cooldown n between identical tasks, find the minimum time to finish all tasks.

[View on LeetCode ↗](#)

APPROACH – The most frequent task dictates idle slots; fill them with other tasks, then count what's left over

```
func leastInterval(_ tasks: [Character], _ n: Int) -> Int {  
    var counts = [Character: Int]()  
    for t in tasks { counts[t, default: 0] += 1 }  
    let maxFreq = counts.values.max()!  
    let maxCount = counts.values.filter { $0 == maxFreq }.count  
    return max(tasks.count, (maxFreq - 1) * (n + 1) + maxCount)  
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 09.06 **Design Twitter** [MEDIUM]

Design a simplified Twitter: post tweets, follow/unfollow users, and fetch the 10 most recent tweets in a user's feed.

[View on LeetCode ↗](#)

APPROACH – Timestamped tweets per user, merged on demand with a max-heap for the news feed


```

class Twitter {

    private var time = 0

    private var tweets = [Int: [(time: Int, id: Int)]]()

    private var following = [Int: Set<Int>]()

    func postTweet(_ userId: Int, _ tweetId: Int) {

        tweets[userId, default: []].append((time, tweetId))

        time += 1

    }

    func getNewsFeed(_ userId: Int) -> [Int] {

        var heap = Heap<(time: Int, id: Int)>(sort: { $0.time > $1.time })

        var users = following[userId] ?? Set<Int>()

        users.insert(userId)

        for u in users {

            for tweet in (tweets[u] ?? []).suffix(10) { heap.push(tweet) }

        }

        var result = [Int]()

        while let t = heap.pop(), result.count < 10 { result.append(t.id) }

        return result

    }

    func follow(_ followerId: Int, _ followeeId: Int) {

        following[followerId, default: []].insert(followeeId)

    }

    func unfollow(_ followerId: Int, _ followeeId: Int) {

        following[followerId]?.remove(followeeId)

    }

}

```

TIME $O(n \log n)$ per feed SPACE $O(n)$

PLATE 09.07 **Find Median from Data Stream** [HARD]

Design a structure supporting addNum and findMedian as numbers stream in.

[View on LeetCode ↗](#)

APPROACH — Two heaps: a max-heap for the lower half, a min-heap for the upper half, kept balanced

```

class MedianFinder {

    private var small = Heap<Int>(sort: >) // lower half, max at top
    private var large = Heap<Int>(sort: <) // upper half, min at top

    func addNum(_ num: Int) {
        small.push(num)
        large.push(small.pop()!)
        if large.count > small.count {
            small.push(large.pop()!)
        }
    }

    func findMedian() -> Double {
        if small.count > large.count { return Double(small.peek!) }
        return Double(small.peek! + large.peek!) / 2.0
    }
}

```

TIME $O(\log n)$ add, $O(1)$ find SPACE $O(n)$

FAMILY 10

Backtracking

Explore, recurse, undo. Every problem here builds a partial solution, recurses into the choices that extend it, and removes the choice on the way back out.

PLATE 10.01 Subsets [MEDIUM]

Given a set of distinct integers, return all possible subsets (the power set).

[View on LeetCode ↗](#)

APPROACH — Backtracking; at each index, either include or skip the current number

```
func subsets(_ nums: [Int]) -> [[Int]] {  
    var result = [[Int]]()  
    var current = [Int]()  
    func backtrack(_ start: Int) {  
        result.append(current)  
        for i in start..  
nums.count {  
            current.append(nums[i])  
            backtrack(i + 1)  
            current.removeLast()  
        }  
    }  
    backtrack(0)  
    return result  
}
```

TIME $O(n \cdot 2^n)$ SPACE $O(n)$

STEP-THROUGH — `nums = [1, 2, 3]` (animated on the companion site)

- Start: `current = []`. Record it — the empty subset always counts.
- Include 1: `current = [1]`. Record it. Then include 2: `current = [1,2]`. Record it. Include 3: `current=[1,2,3]`. Record it.
- Backtrack (remove 3, then 2): try 1 with 3 only → `current=[1,3]`. Record it. Backtrack fully out of 1.
- Move to starting fresh at 2: `current=[2]`, then `[2,3]`. Record both. Then just `[3]`.
- All 8 subsets of a 3-element set recorded: `{}, {1}, {1,2}, {1,2,3}, {1,3}, {2}, {2,3}, {3}`.

PLATE 10.02 Combination Sum [MEDIUM]

Given candidates and a target, return all unique combinations that sum to target (numbers can repeat).

[View on LeetCode ↗](#)

APPROACH — Backtracking that can re-use the current index, since repetition is allowed

```

func combinationSum(_ candidates: [Int], _ target: Int) -> [[Int]] {
    var result = [[Int]]()
    var current = [Int]()

    func backtrack(_ start: Int, _ remaining: Int) {
        if remaining == 0 { result.append(current); return }
        if remaining < 0 { return }
        for i in start..

```

TIME $O(2^n)$ worst case SPACE $O(\text{target})$

PLATE 10.03 **Permutations** [MEDIUM]

Given an array of distinct integers, return all possible permutations.

[View on LeetCode ↗](#)

APPROACH — Backtracking with a “used” tracker, since order matters and every element must appear once

```

func permute(_ nums: [Int]) -> [[Int]] {
    var result = [[Int]]()
    var current = [Int]()
    var used = [Bool](repeating: false, count: nums.count)

    func backtrack() {
        if current.count == nums.count { result.append(current); return }
        for i in 0..

```

TIME $O(n \cdot n!)$ SPACE $O(n)$

PLATE 10.04 **Subsets II** [MEDIUM]

Given an array that may contain duplicates, return all unique subsets.

[View on LeetCode ↗](#)

APPROACH — Sort first, then skip a candidate if it repeats the sibling just tried at this depth

```

func subsetsWithDup(_ nums: [Int]) -> [[Int]] {
    let nums = nums.sorted()
    var result = [[Int]]()
    var current = [Int]()
    func backtrack(_ start: Int) {
        result.append(current)
        for i in start..

```

TIME $O(n \cdot 2^n)$ SPACE $O(n)$

PLATE 10.05 **Combination Sum II** [MEDIUM]

Given candidates (possibly with duplicates) and a target, return all unique combinations that sum to target, using each number once.

[View on LeetCode ↗](#)

APPROACH — Sort, backtrack forward-only, and skip duplicate siblings at the same depth

```

func combinationSum2(_ candidates: [Int], _ target: Int) -> [[Int]] {
    let candidates = candidates.sorted()
    var result = [[Int]]()
    var current = [Int]()
    func backtrack(_ start: Int, _ remaining: Int) {
        if remaining == 0 { result.append(current); return }
        for i in start..

```

TIME $O(2^n)$ SPACE $O(n)$

PLATE 10.06 **Word Search** [MEDIUM]

Given a grid of letters and a word, determine if the word can be formed via adjacent cells.

[View on LeetCode ↗](#)

APPROACH — DFS from every cell, marking visited cells temporarily to avoid reuse

```

func exist(_ board: [[Character]], _ word: String) -> Bool {

    var board = board

    let chars = Array(word)

    let rows = board.count, cols = board[0].count

    func dfs(_ r: Int, _ c: Int, _ i: Int) -> Bool {

        if i == chars.count { return true }

        if r < 0 || r >= rows || c < 0 || c >= cols || board[r][c] != chars[i] { return
false }

        let temp = board[r][c]

        board[r][c] = "#"

        let found = dfs(r+1, c, i+1) || dfs(r-1, c, i+1) || dfs(r, c+1, i+1) || dfs(r,
c-1, i+1)

        board[r][c] = temp

        return found

    }

    for r in 0..

```

TIME $O(\text{rows} \cdot \text{cols} \cdot 4^L)$ SPACE $O(L)$

PLATE 10.07 Palindrome Partitioning [MEDIUM]

Given a string, partition it so every substring is a palindrome. Return all such partitions.

[View on LeetCode ↗](#)

APPROACH — Backtracking; at each start index, try every palindromic prefix as the next cut


```

func partition(_ s: String) -> [[String]] {

    let chars = Array(s)

    var result = [[String]]()

    var current = [String]()

    func isPalindrome(_ l: Int, _ r: Int) -> Bool {

        var l = l, r = r

        while l < r {

            if chars[l] != chars[r] { return false }

            l += 1; r -= 1

        }

        return true

    }

    func backtrack(_ start: Int) {

        if start == chars.count { result.append(current); return }

        for end in start..

```

TIME $O(n \cdot 2^n)$ SPACE $O(n)$

PLATE 10.08 Letter Combinations of a Phone Number [MEDIUM]

Given a string of digits 2-9, return all possible letter combinations the digits could represent (T9 keypad).

[View on LeetCode ↗](#)

APPROACH — Backtracking over each digit's mapped letters

```

func letterCombinations(_ digits: String) -> [String] {
    if digits.isEmpty { return [] }

    let mapping: [Character: String] = [
        "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
        "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"
    ]

    var result = [String]()
    var current = ""

    let digitArr = Array(digits)

    func backtrack(_ index: Int) {
        if index == digitArr.count { result.append(current); return }

        let letters = mapping[digitArr[index]]!

        for letter in letters {
            current.append(letter)
            backtrack(index + 1)
            current.removeLast()
        }
    }

    backtrack(0)

    return result
}

```

TIME $O(4^n \cdot n)$ SPACE $O(n)$

PLATE 10.09 **N-Queens** [HARD]

Place n queens on an $n \times n$ board so no two attack each other. Return all distinct solutions.

[View on LeetCode ↗](#)

APPROACH — Backtracking, one queen per row, tracking occupied columns and both diagonals

```

func solveNQueens(_ n: Int) -> [[String]] {

    var result = [[String]]()

    var cols = Set<Int>(), diag1 = Set<Int>(), diag2 = Set<Int>()

    var placement = [Int](repeating: -1, count: n)

    func backtrack(_ row: Int) {

        if row == n {

            let board = placement.map { col -> String in

                var rowStr = [Character](repeating: ".", count: n)

                rowStr[col] = "Q"

                return String(rowStr)

            }

            result.append(board)

            return

        }

        for col in 0..

```

TIME **$O(n!)$** SPACE **$O(n)$**

FAMILY 11

Graphs

Nodes and edges, explored breadth-first for shortest-hops problems or depth-first for reachability and structure.

PLATE 11.01 Number of Islands [MEDIUM]

Given a grid of '1's (land) and '0's (water), count the number of islands.

[View on LeetCode ↗](#)

APPROACH — DFS flood-fill from every unvisited land cell, sinking it as you go

```
func numIslands(_ grid: [[Character]]) -> Int {  
    var grid = grid  
  
    let rows = grid.count, cols = grid[0].count  
  
    var count = 0  
  
    func dfs(_ r: Int, _ c: Int) {  
        if r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] != "1" { return }  
        grid[r][c] = "0"  
  
        dfs(r+1, c); dfs(r-1, c); dfs(r, c+1); dfs(r, c-1)  
    }  
  
    for r in 0..  
rows {  
        for c in 0..  
cols {  
            if grid[r][c] == "1" {  
                count += 1  
                dfs(r, c)  
            }  
        }  
    }  
  
    return count  
}
```

TIME $O(\text{rows} \cdot \text{cols})$ SPACE $O(\text{rows} \cdot \text{cols})$

PLATE 11.02 Max Area of Island [MEDIUM]

Given a binary grid, return the area of the largest island.

[View on LeetCode ↗](#)

APPROACH — Same flood fill as Number of Islands, but the DFS returns the area it consumes

```

func maxAreaOfIsland(_ grid: [[Int]]) -> Int {
    var grid = grid

    let rows = grid.count, cols = grid[0].count

    var best = 0

    func dfs(_ r: Int, _ c: Int) -> Int {
        if r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] == 0 { return 0 }

        grid[r][c] = 0

        return 1 + dfs(r+1,c) + dfs(r-1,c) + dfs(r,c+1) + dfs(r,c-1)
    }

    for r in 0..

```

TIME $O(\text{rows} \cdot \text{cols})$ SPACE $O(\text{rows} \cdot \text{cols})$

PLATE 11.03 Clone Graph [MEDIUM]

Given a reference node in a connected undirected graph, return a deep copy of the graph.

[View on LeetCode ↗](#)

APPROACH – DFS with a map from original nodes to their clones, to avoid infinite recursion on cycles

```

class GraphNode {

    var val: Int

    var neighbors: [GraphNode]

    init(_ val: Int) { self.val = val; self.neighbors = [] }

}

func cloneGraph(_ node: GraphNode?) -> GraphNode? {

    guard let node = node else { return nil }

    var map = [ObjectIdentifier: GraphNode]()

    func dfs(_ n: GraphNode) -> GraphNode {

        if let existing = map[ObjectIdentifier(n)] { return existing }

        let copy = GraphNode(n.val)

        map[ObjectIdentifier(n)] = copy

        for neighbor in n.neighbors {

            copy.neighbors.append(dfs(neighbor))

        }

        return copy

    }

    return dfs(node)

}

```

TIME $O(V + E)$ SPACE $O(V)$

PLATE 11.04 Islands and Treasure [MEDIUM]

Given a grid of gates (0), walls (-1), and infinite cells, fill each infinite cell with its distance to the nearest gate.

[View on LeetCode ↗](#)

APPROACH — Multi-source BFS starting from every gate simultaneously

```

func islandsAndTreasure(_ grid: inout [[Int]]) {

    let rows = grid.count, cols = grid[0].count

    let INF = 2147483647

    var queue = [(Int, Int)]()

    for r in 0..

```

TIME $O(\text{rows} \cdot \text{cols})$ SPACE $O(\text{rows} \cdot \text{cols})$

PLATE 11.05 **Rotting Oranges** [MEDIUM]

Given a grid of fresh, rotten, and empty cells, find the minutes until no fresh orange remains (or -1 if impossible).

[View on LeetCode ↗](#)

APPROACH — Multi-source BFS from all rotten oranges, one minute per wave

```

func orangesRotting(_ grid: [[Int]]) -> Int {
    var grid = grid

    let rows = grid.count, cols = grid[0].count
    var queue = [(Int, Int)]()
    var fresh = 0

    for r in 0..

```

TIME $O(\text{rows} \cdot \text{cols})$ SPACE $O(\text{rows} \cdot \text{cols})$

PLATE 11.06 Pacific Atlantic Water Flow [MEDIUM]

Given a height grid bordered by the Pacific (top/left) and Atlantic (bottom/right), find cells from which water can reach both oceans.

[View on LeetCode ↗](#)

APPROACH – DFS inward from each ocean's border (flow uphill), then intersect the two reachable sets

```
func pacificAtlantic(_ heights: [[Int]]) -> [[Int]] {
    let rows = heights.count, cols = heights[0].count
    var pacific = Set<Int>(), atlantic = Set<Int>()
    let dirs = [(1,0), (-1,0), (0,1), (0,-1)]

    func dfs(_ r: Int, _ c: Int, _ visited: inout Set<Int>, _ prevHeight: Int) {
        let key = r * cols + c
        if r < 0 || r >= rows || c < 0 || c >= cols || visited.contains(key) ||
heights[r][c] < prevHeight { return }

        visited.insert(key)
        for (dr, dc) in dirs { dfs(r + dr, c + dc, &visited, heights[r][c]) }
    }

    for c in 0..
```

TIME $O(\text{rows} \cdot \text{cols})$ SPACE $O(\text{rows} \cdot \text{cols})$

PLATE 11.07 **Surrounded Regions** [MEDIUM]

Given a board of 'X' and 'O', flip every 'O' that is not connected to the border to 'X'.

[View on LeetCode ↗](#)

APPROACH – Mark border-connected 'O's safe via DFS first, then flip everything else

```
func solve(_ board: inout [[Character]]) {  
    let rows = board.count, cols = board[0].count  
  
    func dfs(_ r: Int, _ c: Int) {  
        if r < 0 || r >= rows || c < 0 || c >= cols || board[r][c] != "O" { return }  
        board[r][c] = "T"  
        dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1)  
    }  
  
    for r in 0..  
rows {  
        dfs(r, 0); dfs(r, cols - 1)  
    }  
  
    for c in 0..  
cols {  
        dfs(0, c); dfs(rows - 1, c)  
    }  
  
    for r in 0..  
rows {  
        for c in 0..  
cols {  
            if board[r][c] == "O" { board[r][c] = "X" }  
            else if board[r][c] == "T" { board[r][c] = "O" }  
        }  
    }  
}
```

TIME $O(\text{rows} \cdot \text{cols})$ SPACE $O(\text{rows} \cdot \text{cols})$

PLATE 11.08 **Course Schedule** [MEDIUM]

Given numCourses and prerequisite pairs, determine if it's possible to finish all courses.

[View on LeetCode ↗](#)

APPROACH – Detect a cycle in the prerequisite graph via DFS with a 3-state visited marker

```

func canFinish(_ numCourses: Int, _ prerequisites: [[Int]]) -> Bool {

    var graph = [[Int]](repeating: [], count: numCourses)

    for p in prerequisites { graph[p[0]].append(p[1]) }

    var state = [Int](repeating: 0, count: numCourses) // 0=unvisited, 1=visiting,
2=done

    func dfs(_ course: Int) -> Bool {

        if state[course] == 1 { return false }

        if state[course] == 2 { return true }

        state[course] = 1

        for next in graph[course] {

            if !dfs(next) { return false }

        }

        state[course] = 2

        return true

    }

    for course in 0..

```

TIME $O(V + E)$ SPACE $O(V + E)$

PLATE 11.09 **Course Schedule II** [MEDIUM]

Return a valid order to finish all courses given prerequisites, or an empty array if impossible.

[View on LeetCode ↗](#)

APPROACH — Kahn's algorithm: topological sort via indegree-0 queue

```

func findOrder(_ numCourses: Int, _ prerequisites: [[Int]]) -> [Int] {

    var graph = [[Int]](repeating: [], count: numCourses)

    var indegree = [Int](repeating: 0, count: numCourses)

    for p in prerequisites {

        graph[p[1]].append(p[0])

        indegree[p[0]] += 1

    }

    var queue = (0..

```

TIME $O(V + E)$ SPACE $O(V + E)$

PLATE 11.10 **Graph Valid Tree** [MEDIUM]

Given n nodes and a list of edges, determine if they form a valid tree.

[View on LeetCode ↗](#)

APPROACH — A tree needs exactly $n-1$ edges and full connectivity — check both with a BFS

```

func validTree(_ n: Int, _ edges: [[Int]]) -> Bool {

    if edges.count != n - 1 { return false }

    var graph = [[Int]](repeating: [], count: n)

    for e in edges {

        graph[e[0]].append(e[1])

        graph[e[1]].append(e[0])

    }

    var visited = Set<Int>()

    var queue = [0]

    visited.insert(0)

    var index = 0

    while index < queue.count {

        let node = queue[index]; index += 1

        for neighbor in graph[node] {

            if !visited.contains(neighbor) {

                visited.insert(neighbor)

                queue.append(neighbor)

            }

        }

    }

    return visited.count == n

}

```

TIME $O(V + E)$ SPACE $O(V + E)$

PLATE 11.11 **Number of Connected Components in an Undirected Graph** [MEDIUM]

Given n nodes and a list of edges, count the number of connected components.

[View on LeetCode ↗](#)

APPROACH — Union-Find; each successful union merges two components into one

```

func countComponents(_ n: Int, _ edges: [[Int]]) -> Int {
    var parent = Array(0..

```

TIME $O(E \cdot \alpha(n))$ SPACE $O(n)$

PLATE 11.12 Redundant Connection [MEDIUM]

Given a tree with one extra edge added (creating a cycle), find the edge that can be removed.

[View on LeetCode ↗](#)

APPROACH – Union-Find; the first edge that connects two already-connected nodes is the redundant one

```

func findRedundantConnection(_ edges: [[Int]]) -> [Int] {
    var parent = Array(0...edges.count)

    func find(_ x: Int) -> Int {
        var x = x
        while parent[x] != x { parent[x] = parent[parent[x]]; x = parent[x] }
        return x
    }

    for e in edges {
        let a = find(e[0]), b = find(e[1])
        if a == b { return e }
        parent[a] = b
    }

    return []
}

```

TIME $O(E \cdot \alpha(n))$ SPACE $O(n)$

PLATE 11.13 **Word Ladder** **[HARD]**

Given beginWord, endWord, and a word list, find the length of the shortest transformation sequence changing one letter at a time.

[View on LeetCode ↗](#)

APPROACH — BFS over the implicit graph of one-letter-different words

```

func ladderLength(_ beginWord: String, _ endWord: String, _ wordList: [String]) -> Int {
    var wordSet = Set(wordList)

    guard wordSet.contains(endWord) else { return 0 }

    var queue = [beginWord]

    var steps = 1

    let letters = Array("abcdefghijklmnopqrstuvwxyz")

    while !queue.isEmpty {
        var next = [String]()

        for word in queue {
            if word == endWord { return steps }

            var chars = Array(word)

            for i in 0..

```

TIME $O(n \cdot 26 \cdot L)$

SPACE $O(n)$

FAMILY 12

Advanced Graphs

Weighted edges bring in Dijkstra, Prim's, Bellman-Ford, and topological order — shortest paths and minimum spanning structures.

PLATE 12.01 Reconstruct Itinerary [HARD]

Given airline tickets as [from, to] pairs, reconstruct the itinerary in order, starting from “JFK,” using every ticket exactly once, lexicographically smallest.

[View on LeetCode ↗](#)

APPROACH — Hierholzer's algorithm: greedily follow the smallest destination, backtrack (Eulerian path) on dead ends

```
func findItinerary(_ tickets: [[String]]) -> [String] {
    var graph = [String: [String]]()
    for t in tickets.sorted(by: { $0[1] < $1[1] }) {
        graph[t[0], default: []].append(t[1])
    }
    var route = [String]()

    func dfs(_ airport: String) {
        while let next = graph[airport]?.first {
            graph[airport]?.removeFirst()
            dfs(next)
        }
        route.append(airport)
    }

    dfs("JFK")

    return route.reversed()
}
```

TIME $O(E \log E)$ SPACE $O(E)$

PLATE 12.02 Min Cost to Connect All Points [MEDIUM]

Given points on a plane, find the minimum cost to connect them all (cost = Manhattan distance).

[View on LeetCode ↗](#)

APPROACH — Prim's algorithm: grow a minimum spanning tree one nearest point at a time

```

func minCostConnectPoints(_ points: [[Int]]) -> Int {
    let n = points.count
    var visited = [Bool](repeating: false, count: n)
    var minDist = [Int](repeating: Int.max, count: n)
    minDist[0] = 0
    var total = 0

    for _ in 0..

```

TIME $O(n^2)$ SPACE $O(n)$

PLATE 12.03 Network Delay Time [MEDIUM]

Given a network of weighted directed edges and a source node k , find the time for all nodes to receive a signal, or -1 if impossible.

[View on LeetCode ↗](#)

APPROACH — Dijkstra's algorithm with a min-heap

```

func networkDelayTime(_ times: [[Int]], _ n: Int, _ k: Int) -> Int {
    var graph = [Int: [(Int, Int)]]()
    for t in times { graph[t[0], default: []].append((t[1], t[2])) }

    var dist = [Int: Int]()
    var heap = Heap<(node: Int, time: Int)>(sort: { $0.time < $1.time })
    heap.push((k, 0))

    while let (node, time) = heap.pop() {
        if dist[node] != nil { continue }
        dist[node] = time
        for (next, weight) in graph[node] ?? [] {
            if dist[next] == nil { heap.push((next, time + weight)) }
        }
    }

    return dist.count == n ? dist.values.max()! : -1
}

```

TIME $O(E \log V)$ SPACE $O(V + E)$

PLATE 12.04 **Swim in Rising Water** [HARD]

Given an elevation grid, find the minimum time to swim from top-left to bottom-right as water rises.

[View on LeetCode ↗](#)

APPROACH — Dijkstra-style min-heap, where the “cost” of a path is its maximum elevation along the way

```

func swimInWater(_ grid: [[Int]]) -> Int {
    let n = grid.count
    var visited = Set<Int>()
    var heap = Heap<(time: Int, r: Int, c: Int)>(sort: { $0.time < $1.time })
    heap.push((grid[0][0], 0, 0))
    visited.insert(0)
    let dirs = [(1,0), (-1,0), (0,1), (0,-1)]

    while let (time, r, c) = heap.pop() {
        if r == n - 1 && c == n - 1 { return time }
        for (dr, dc) in dirs {
            let nr = r + dr, nc = c + dc
            let key = nr * n + nc
            if nr >= 0 && nr < n && nc >= 0 && nc < n && !visited.contains(key) {
                visited.insert(key)
                heap.push((max(time, grid[nr][nc]), nr, nc))
            }
        }
    }

    return -1
}

```

TIME $O(n^2 \log n)$ SPACE $O(n^2)$

PLATE 12.05 Alien Dictionary [HARD]

Given a sorted list of words from an alien language, derive a valid character ordering.

[View on LeetCode ↗](#)

APPROACH — Build a precedence graph from adjacent word comparisons, then topologically sort it

```

func alienOrder(_ words: [String]) -> String {

    var graph = [Character: Set<Character>]()

    var indegree = [Character: Int]()

    for word in words {

        for c in word { indegree[c] = indegree[c] ?? 0; graph[c] = graph[c] ?? [] }

    }

    for i in 0..

```

TIME **O(total characters)** SPACE **O(1)**

PLATE 12.06 **Cheapest Flights Within K Stops** **[MEDIUM]**

Given flights with prices, find the cheapest price from src to dst with at most k stops.

[View on LeetCode ↗](#)

APPROACH – Bellman-Ford relaxation, limited to k+1 rounds so extra stops can't sneak in

```
func findCheapestPrice(_ n: Int, _ flights: [[Int]], _ src: Int, _ dst: Int, _ k: Int) -> Int {  
    var dist = [Int](repeating: Int.max, count: n)  
  
    dist[src] = 0  
  
    for _ in 0...k {  
        var next = dist  
  
        for f in flights {  
            let (u, v, w) = (f[0], f[1], f[2])  
  
            if dist[u] != Int.max && dist[u] + w < next[v] {  
                next[v] = dist[u] + w  
            }  
        }  
  
        dist = next  
    }  
  
    return dist[dst] == Int.max ? -1 : dist[dst]  
}
```

TIME **O(k · E)** SPACE **O(n)**

FAMILY 13

1-D Dynamic Programming

Each state depends on a constant number of previous states along a single dimension — often collapsible from an array down to $O(1)$ space.

PLATE 13.01 Climbing Stairs [EASY]

You can climb 1 or 2 steps at a time. Given n steps, how many distinct ways can you climb to the top?

[View on LeetCode ↗](#)

APPROACH — Fibonacci-shaped recurrence, collapsed to $O(1)$ space

```
func climbStairs(_ n: Int) -> Int {  
    if n <= 2 { return n }  
  
    var prev = 1, curr = 2  
  
    for _ in 3...n {  
        (prev, curr) = (curr, prev + curr)  
    }  
  
    return curr  
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 13.02 Min Cost Climbing Stairs [EASY]

Given a cost array, find the minimum cost to reach the top, starting from step 0 or 1, moving 1 or 2 steps at a time.

[View on LeetCode ↗](#)

APPROACH — $dp[i]$ = cost to have already reached step i ; roll two variables forward

```
func minCostClimbingStairs(_ cost: [Int]) -> Int {  
    var prev = 0, curr = 0  
  
    for i in 2...cost.count {  
        let next = min(curr + cost[i - 1], prev + cost[i - 2])  
  
        prev = curr  
        curr = next  
    }  
  
    return curr  
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 13.03 House Robber [MEDIUM]

Given houses with money, find the maximum amount you can rob without robbing two adjacent houses.

[View on LeetCode ↗](#)

APPROACH — $dp[i]$ = max(skip this house, rob it + best from two houses back), collapsed to $O(1)$ space

```

func rob(_ nums: [Int]) -> Int {
    var prev = 0, curr = 0
    for n in nums {
        (prev, curr) = (curr, max(curr, prev + n))
    }
    return curr
}

```

TIME $O(n)$ SPACE $O(1)$

STEP-THROUGH – `nums = [2,7,9,3,1]` (animated on the companion site)

- House 2: best so far = $\max(0, 0+2) = 2$.
- House 7: best = $\max(2, 0+7) = 7$.
- House 9: best = $\max(7, 2+9) = 11$.
- House 3: best = $\max(11, 7+3) = 11$ — robbing this house doesn't help.
- House 1: best = $\max(11, 11+1) = 12$. Maximum haul = 12 (houses at index 0, 2, 4).

PLATE 13.04 **House Robber II** [MEDIUM]

Same as House Robber, but the houses are arranged in a circle (first and last are adjacent).

[View on LeetCode ↗](#)

APPROACH – Run House Robber twice, once excluding the first house and once excluding the last, take the max

```

func robII(_ nums: [Int]) -> Int {
    if nums.count == 1 { return nums[0] }

    func robLine(_ nums: ArraySlice<Int>) -> Int {
        var prev = 0, curr = 0
        for n in nums { (prev, curr) = (curr, max(curr, prev + n)) }
        return curr
    }

    return max(robLine(nums.dropLast()), robLine(nums.dropFirst()))
}

```

TIME $O(n)$ SPACE $O(1)$

PLATE 13.05 **Longest Palindromic Substring** [MEDIUM]

Given a string, return its longest palindromic substring.

[View on LeetCode ↗](#)

APPROACH – Expand around every center (odd and even length), track the best window


```

func longestPalindrome(_ s: String) -> String {

    let chars = Array(s)

    var start = 0, maxLen = 0

    func expand(_ l: Int, _ r: Int) {

        var l = l, r = r

        while l >= 0 && r < chars.count && chars[l] == chars[r] {

            if r - l + 1 > maxLen {

                start = l

                maxLen = r - l + 1

            }

            l -= 1; r += 1

        }

    }

    for i in 0..

```

TIME $O(n^2)$ SPACE $O(1)$

PLATE 13.06 **Palindromic Substrings** [MEDIUM]

Given a string, count how many substrings of it are palindromes.

[View on LeetCode ↗](#)

APPROACH — Same **expand-around-center technique**, counting every valid expansion

```

func countSubstrings(_ s: String) -> Int {

    let chars = Array(s)

    var count = 0

    func expand(_ l: Int, _ r: Int) {

        var l = l, r = r

        while l >= 0 && r < chars.count && chars[l] == chars[r] {

            count += 1

            l -= 1; r += 1

        }

    }

    for i in 0..

```

TIME $O(n^2)$ SPACE $O(1)$

PLATE 13.07 **Decode Ways** [MEDIUM]

Given a digit string that could map to letters (1='A' ... 26='Z'), count how many ways it can be decoded.

[View on LeetCode ↗](#)

APPROACH – $dp[i]$ = decodings using a single digit plus decodings using the last two digits

```

func numDecodings(_ s: String) -> Int {

    let chars = Array(s)

    guard chars.first != "0" else { return 0 }

    var prev = 1, curr = 1

    for i in 1..  
chars.count {

        var next = 0

        if chars[i] != "0" { next += curr }

        let twoDigit = Int(String(chars[i-1..  
i]))!

        if twoDigit >= 10 && twoDigit <= 26 { next += prev }

        prev = curr

        curr = next

    }

    return curr

}

```

TIME **O(n)** SPACE **O(1)**

PLATE 13.08 **Coin Change** [MEDIUM]

Given coin denominations and a target amount, find the fewest coins needed to make that amount (or -1).

[View on LeetCode ↗](#)

APPROACH — Bottom-up DP over amounts from 0 to target

```

func coinChange(_ coins: [Int], _ amount: Int) -> Int {

    var dp = [Int](repeating: amount + 1, count: amount + 1)

    dp[0] = 0

    for a in 1..  
amount {

        for coin in coins where coin <= a {

            dp[a] = min(dp[a], dp[a - coin] + 1)

        }

    }

    return dp[amount] > amount ? -1 : dp[amount]

}

```

TIME **O(amount · coins)** SPACE **O(amount)**

PLATE 13.09 **Maximum Product Subarray** [MEDIUM]

Given an integer array, find the contiguous subarray with the largest product.

[View on LeetCode ↗](#)

APPROACH — Track both running max and min (a negative number can flip min into the new max)

```

func maxProduct(_ nums: [Int]) -> Int {
    var result = nums[0], curMax = nums[0], curMin = nums[0]

    for n in nums.dropFirst() {
        let candidates = [n, curMax * n, curMin * n]

        curMax = candidates.max()!
        curMin = candidates.min()!

        result = max(result, curMax)
    }

    return result
}

```

TIME $O(n)$ SPACE $O(1)$

PLATE 13.10 **Word Break** [MEDIUM]

Given a string and a dictionary of words, determine if the string can be segmented into dictionary words.

[View on LeetCode ↗](#)

APPROACH — $dp[i]$ = true if $s[0..<i]$ can be segmented; check every earlier split point

```

func wordBreak(_ s: String, _ wordDict: [String]) -> Bool {
    let words = Set(wordDict)
    let chars = Array(s)
    var dp = [Bool](repeating: false, count: chars.count + 1)
    dp[0] = true
    for i in 1...chars.count {
        for j in 0.. {
            if dp[j] && words.contains(String(chars[j..])) {
                dp[i] = true
                break
            }
        }
    }
    return dp[chars.count]
}

```

TIME $O(n^2)$ SPACE $O(n)$

PLATE 13.11 **Longest Increasing Subsequence** [MEDIUM]

Given an integer array, return the length of the longest strictly increasing subsequence.

[View on LeetCode ↗](#)

APPROACH — Patience sorting: maintain the smallest possible tail for each subsequence length, binary search to place each number

```

func lengthOfLIS(_ nums: [Int]) -> Int {
    var tails = [Int]()
    for n in nums {
        var lo = 0, hi = tails.count
        while lo < hi {
            let mid = lo + (hi - lo) / 2
            if tails[mid] < n { lo = mid + 1 } else { hi = mid }
        }
        if lo == tails.count { tails.append(n) } else { tails[lo] = n }
    }
    return tails.count
}

```

TIME $O(n \log n)$ SPACE $O(n)$

PLATE 13.12 Partition Equal Subset Sum [MEDIUM]

Given an array of positive integers, determine if it can be split into two subsets with equal sums.

[View on LeetCode ↗](#)

APPROACH – Subset-sum DP: track every achievable sum up to total/2 using a Set

```

func canPartition(_ nums: [Int]) -> Bool {
    let total = nums.reduce(0, +)
    if total % 2 != 0 { return false }
    let target = total / 2
    var dp = Set<Int>([0])
    for n in nums {
        var next = dp
        for s in dp where s + n <= target { next.insert(s + n) }
        dp = next
        if dp.contains(target) { return true }
    }
    return dp.contains(target)
}

```

TIME $O(n \cdot \text{target})$ SPACE $O(\text{target})$

FAMILY 14

2-D Dynamic Programming

Two sequences or two dimensions of choice mean a grid of subproblems, typically filled row by row.

PLATE 14.01 Unique Paths [MEDIUM]

A robot starts at the top-left of an $m \times n$ grid and can only move right or down. Count the paths to the bottom-right.

[View on LeetCode ↗](#)

APPROACH — dp over a single row, since each cell only depends on the cell above and to the left

```
func uniquePaths(_ m: Int, _ n: Int) -> Int {  
    var row = [Int](repeating: 1, count: n)  
  
    for _ in 1.. $m$  {  
        for c in 1.. $n$  {  
            row[c] += row[c - 1]  
        }  
    }  
  
    return row[n - 1]  
}
```

TIME $O(m \cdot n)$ SPACE $O(n)$

PLATE 14.02 Longest Common Subsequence [MEDIUM]

Given two strings, return the length of their longest common subsequence.

[View on LeetCode ↗](#)

APPROACH — 2-D grid DP; match extends the diagonal, mismatch takes the best of skipping either character

```
func longestCommonSubsequence(_ text1: String, _ text2: String) -> Int {  
    let a = Array(text1), b = Array(text2)  
  
    var dp = [[Int]](repeating: [Int](repeating: 0, count: b.count + 1), count: a.count + 1)  
  
    for i in 1...a.count {  
        for j in 1...b.count {  
            if a[i-1] == b[j-1] { dp[i][j] = dp[i-1][j-1] + 1 }  
            else { dp[i][j] = max(dp[i-1][j], dp[i][j-1]) }  
        }  
    }  
  
    return dp[a.count][b.count]  
}
```

TIME $O(m \cdot n)$ SPACE $O(m \cdot n)$

PLATE 14.03 Best Time to Buy and Sell Stock with Cooldown [MEDIUM]

Given prices, maximize profit with unlimited transactions, but a 1-day cooldown after selling.

[View on LeetCode ↗](#)

APPROACH – Track three running states: held, sold today, and resting (able to buy)

```
func maxProfitCooldown(_ prices: [Int]) -> Int {  
    var sold = Int.min, held = Int.min, rest = 0  
    for price in prices {  
        let prevSold = sold  
        sold = held + price  
        held = max(held, rest - price)  
        rest = max(rest, prevSold)  
    }  
    return max(sold, rest)  
}
```

TIME **O(n)** SPACE **O(1)**

PLATE 14.04 **Coin Change II** [MEDIUM]

Given coins and an amount, count the number of distinct combinations that make up that amount.

[View on LeetCode ↗](#)

APPROACH – dp over amounts, processing one coin denomination fully before the next (avoids counting order as distinct)

```
func change(_ amount: Int, _ coins: [Int]) -> Int {  
    var dp = [Int](repeating: 0, count: amount + 1)  
    dp[0] = 1  
    for coin in coins {  
        for a in coin...amount {  
            dp[a] += dp[a - coin]  
        }  
    }  
    return dp[amount]  
}
```

TIME **O(amount · coins)** SPACE **O(amount)**

PLATE 14.05 **Target Sum** [MEDIUM]

Given numbers and a target, count the ways to assign + or – to each number so they sum to target.

[View on LeetCode ↗](#)

APPROACH – Track achievable sums and their counts in a running dictionary

```

func findTargetSumWays(_ nums: [Int], _ target: Int) -> Int {

    var dp = [Int: Int]()

    dp[0] = 1

    for n in nums {

        var next = [Int: Int]()

        for (sum, count) in dp {

            next[sum + n, default: 0] += count

            next[sum - n, default: 0] += count

        }

        dp = next

    }

    return dp[target] ?? 0

}

```

TIME $O(n \cdot \text{sumRange})$ SPACE $O(\text{sumRange})$

PLATE 14.06 Interleaving String [MEDIUM]

Given s1, s2, s3, determine if s3 is formed by interleaving s1 and s2.

[View on LeetCode ↗](#)

APPROACH – 2-D DP; `dp[i][j]` means the first `i+j` characters of `s3` can be formed from `s1[..i]` and `s2[..j]`

```

func isInterleave(_ s1: String, _ s2: String, _ s3: String) -> Bool {

    let a = Array(s1), b = Array(s2), c = Array(s3)

    guard a.count + b.count == c.count else { return false }

    var dp = [[Bool]](repeating: [Bool](repeating: false, count: b.count + 1), count: a.count + 1)

    dp[0][0] = true

    for i in 0...a.count {

        for j in 0...b.count {

            if i > 0 && dp[i-1][j] && a[i-1] == c[i+j-1] { dp[i][j] = true }

            if j > 0 && dp[i][j-1] && b[j-1] == c[i+j-1] { dp[i][j] = true }

        }

    }

    return dp[a.count][b.count]

}

```

TIME $O(m \cdot n)$ SPACE $O(m \cdot n)$

PLATE 14.07 Longest Increasing Path in a Matrix [HARD]

Given a matrix, find the length of the longest strictly increasing path (moving up/down/left/right).

[View on LeetCode ↗](#)

APPROACH – DFS with memoization; each cell's answer only depends on strictly larger neighbors

```
func longestIncreasingPath(_ matrix: [[Int]]) -> Int {  
    let rows = matrix.count, cols = matrix[0].count  
    var memo = [[Int]](repeating: [Int](repeating: 0, count: cols), count: rows)  
    let dirs = [(1,0), (-1,0), (0,1), (0,-1)]  
  
    func dfs(_ r: Int, _ c: Int) -> Int {  
        if memo[r][c] != 0 { return memo[r][c] }  
        var best = 1  
        for (dr, dc) in dirs {  
            let nr = r + dr, nc = c + dc  
            if nr >= 0 && nr < rows && nc >= 0 && nc < cols && matrix[nr][nc] > matrix[r]  
[c] {  
                best = max(best, 1 + dfs(nr, nc))  
            }  
        }  
        memo[r][c] = best  
        return best  
    }  
  
    var result = 0  
    for r in 0..  
rows {  
        for c in 0..  
cols {  
            result = max(result, dfs(r, c))  
        }  
    }  
    return result  
}
```

TIME $O(\text{rows} \cdot \text{cols})$ SPACE $O(\text{rows} \cdot \text{cols})$

PLATE 14.08 **Distinct Subsequences** [HARD]

Given strings s and t , count the number of distinct subsequences of s that equal t .

[View on LeetCode ↗](#)

APPROACH – 2-D DP; a match can either be used or skipped, a mismatch must be skipped

```

func numDistinct(_ s: String, _ t: String) -> Int {
    let a = Array(s), b = Array(t)

    var dp = [[Int]](repeating: [Int](repeating: 0, count: b.count + 1), count: a.count + 1)

    for i in 0...a.count { dp[i][0] = 1 }

    for i in 1...a.count {
        for j in 1...b.count {
            dp[i][j] = dp[i-1][j]

            if a[i-1] == b[j-1] { dp[i][j] += dp[i-1][j-1] }
        }
    }

    return dp[a.count][b.count]
}

```

TIME $O(m \cdot n)$ SPACE $O(m \cdot n)$

PLATE 14.09 **Edit Distance** [MEDIUM]

Given two words, find the minimum number of insertions, deletions, or substitutions to turn one into the other.

[View on LeetCode ↗](#)

APPROACH — Classic 2-D DP: match keeps the diagonal, mismatch takes 1 + best of insert/delete/replace

```

func minDistance(_ word1: String, _ word2: String) -> Int {
    let a = Array(word1), b = Array(word2)

    var dp = [[Int]](repeating: [Int](repeating: 0, count: b.count + 1), count: a.count + 1)

    for i in 0...a.count { dp[i][0] = i }
    for j in 0...b.count { dp[0][j] = j }

    for i in 1...a.count {
        for j in 1...b.count {
            if a[i-1] == b[j-1] { dp[i][j] = dp[i-1][j-1] }
            else { dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) }
        }
    }

    return dp[a.count][b.count]
}

```

TIME $O(m \cdot n)$ SPACE $O(m \cdot n)$

PLATE 14.10 **Burst Balloons** [HARD]

Given balloons with values, find the maximum coins from bursting them all (bursting i gives $\text{left} \times i \times \text{right}$ coins).

[View on LeetCode ↗](#)

APPROACH – Interval DP; think in reverse – choose the last balloon to burst in each interval

```
func maxCoins(_ nums: [Int]) -> Int {
    let balloons = [1] + nums + [1]

    let n = balloons.count

    var dp = [[Int]](repeating: [Int](repeating: 0, count: n), count: n)

    for length in 2..
```

TIME $O(n^3)$ SPACE $O(n^2)$

PLATE 14.11 **Regular Expression Matching** [HARD]

Implement regex matching supporting '.' (any character) and '*' (zero or more of the preceding element).

[View on LeetCode ↗](#)

APPROACH – 2-D DP over (s, p) prefixes, with special handling for '*'

```

func isMatch(_ s: String, _ p: String) -> Bool {

    let a = Array(s), b = Array(p)

    var dp = [[Bool]](repeating: [Bool](repeating: false, count: b.count + 1), count:
a.count + 1)

    dp[0][0] = true

    for j in 1...b.count {

        if b[j-1] == "*" { dp[0][j] = dp[0][j-2] }

    }

    for i in 1...a.count {

        for j in 1...b.count {

            if b[j-1] == "." || b[j-1] == a[i-1] {

                dp[i][j] = dp[i-1][j-1]

            } else if b[j-1] == "*" {

                dp[i][j] = dp[i][j-2]

                if b[j-2] == "." || b[j-2] == a[i-1] {

                    dp[i][j] = dp[i][j] || dp[i-1][j]

                }

            }

        }

    }

    return dp[a.count][b.count]

}

```

TIME $O(m \cdot n)$ SPACE $O(m \cdot n)$

FAMILY 15

Greedy

Committing to the locally best choice at each step, and trusting (or proving) that choice never costs you the global optimum.

PLATE 15.01 Maximum Subarray [MEDIUM]

Given an integer array, find the contiguous subarray with the largest sum.

[View on LeetCode ↗](#)

APPROACH — Kadane's algorithm: extend the current run or restart at the current element, whichever is bigger

```
func maxSubArray(_ nums: [Int]) -> Int {  
    var best = nums[0], current = nums[0]  
    for n in nums.dropFirst() {  
        current = max(n, current + n)  
        best = max(best, current)  
    }  
    return best  
}
```

TIME $O(n)$ SPACE $O(1)$

STEP-THROUGH — `nums = [-2,1,-3,4,-1,2,1,-5,4]` (animated on the companion site)

- Start: `current = best = -2`.
- `n=4`: `current = max(4, -2+4) = 4`. `best = 4` — a fresh start beats dragging along the negative prefix.
- `n=2, n=1`: current keeps growing, 5 then 6. `best = 6`.
- `n=-5, n=4`: current dips to 1, then rises to 5 — never beats 6.
- Best subarray sum = 6, from `[4, -1, 2, 1]`.

PLATE 15.02 Jump Game [MEDIUM]

Given an array where each element is your max jump length from that position, determine if you can reach the last index.

[View on LeetCode ↗](#)

APPROACH — Greedily track the farthest reachable index while scanning left to right

```
func canJump(_ nums: [Int]) -> Bool {  
    var reach = 0  
    for i in 0..nums.count {  
        if i > reach { return false }  
        reach = max(reach, i + nums[i])  
    }  
    return true  
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 15.03 **Jump Game II** [MEDIUM]

Given the same setup as Jump Game, return the minimum number of jumps to reach the last index.

[View on LeetCode ↗](#)

APPROACH – Greedy BFS-by-levels: jump only when you've exhausted the current level's reach

```
func jump(_ nums: [Int]) -> Int {
    var jumps = 0, currentEnd = 0, farthest = 0
    for i in 0..
```

TIME $O(n)$ SPACE $O(1)$

PLATE 15.04 **Gas Station** [MEDIUM]

Given gas and cost at each station in a circular route, find the starting station that allows completing the circuit, or -1.

[View on LeetCode ↗](#)

APPROACH – If total gas $\geq$ total cost, a solution exists; the greedy restart point is right after any deficit

```
func canCompleteCircuit(_ gas: [Int], _ cost: [Int]) -> Int {
    var total = 0, tank = 0, start = 0
    for i in 0.<gas.count {
        let diff = gas[i] - cost[i]
        total += diff
        tank += diff
        if tank < 0 {
            start = i + 1
            tank = 0
        }
    }
    return total >= 0 ? start : -1
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 15.05 **Hand of Straights** [MEDIUM]

Given a hand of cards and a group size, determine if the hand can be rearranged into groups of consecutive cards.

[View on LeetCode ↗](#)

APPROACH – Greedily build each group starting from the smallest remaining card

```
func isNStraightHand(_ hand: [Int], _ groupSize: Int) -> Bool {  
    if hand.count % groupSize != 0 { return false }  
    var counts = [Int: Int]()  
    for card in hand { counts[card, default: 0] += 1 }  
    let sortedKeys = counts.keys.sorted()  
    for key in sortedKeys {  
        let count = counts[key] ?? 0  
        if count == 0 { continue }  
        for next in key.. $(key + groupSize)$  {  
            let available = counts[next] ?? 0  
            if available < count { return false }  
            counts[next] = available - count  
        }  
    }  
    return true  
}
```

TIME $O(n \log n)$ SPACE $O(n)$

PLATE 15.06 **Merge Triplets to Form Target Triplet** [MEDIUM]

Given triplets and a target triplet, determine if you can pick some triplets and merge them (taking the max per position) to form the target.

[View on LeetCode ↗](#)

APPROACH – Only consider triplets that don't exceed the target in any position; track which target positions become achievable

```

func mergeTriplets(_ triplets: [[Int]], _ target: [Int]) -> Bool {
    var achievable = Set<Int>()

    for t in triplets {
        if t[0] <= target[0] && t[1] <= target[1] && t[2] <= target[2] {
            for i in 0..<3 where t[i] == target[i] { achievable.insert(i) }
        }
    }

    return achievable.count == 3
}

```

TIME **O(n)** SPACE **O(1)**

PLATE 15.07 Partition Labels [MEDIUM]

Given a string, partition it into as many parts as possible so each letter appears in at most one part.

[View on LeetCode ↗](#)

APPROACH – For each character track its last occurrence; extend the current partition's end to cover every character seen so far

```

func partitionLabels(_ s: String) -> [Int] {
    let chars = Array(s)
    var lastIndex = [Character: Int]()

    for (i, c) in chars.enumerated() { lastIndex[c] = i }

    var result = [Int]()
    var start = 0, end = 0

    for (i, c) in chars.enumerated() {
        end = max(end, lastIndex[c]!)

        if i == end {
            result.append(end - start + 1)
            start = i + 1
        }
    }

    return result
}

```

TIME **O(n)** SPACE **O(1)**

PLATE 15.08 Valid Parenthesis String [MEDIUM]

Given a string with '(', ')', and '*' (wildcard for '(', ')', or empty), determine if it can be valid.

[View on LeetCode ↗](#)

APPROACH – Track the range [lo, hi] of possible open-paren counts as you scan


```
func checkValidString(_ s: String) -> Bool {  
    var lo = 0, hi = 0  
  
    for c in s {  
        if c == "(" { lo += 1; hi += 1 }  
        else if c == ")" { lo -= 1; hi -= 1 }  
        else { lo -= 1; hi += 1 }  
        if hi < 0 { return false }  
        lo = max(lo, 0)  
    }  
    return lo == 0  
}
```

TIME **$O(n)$** SPACE **$O(1)$**

FAMILY 16

Intervals

Sort by start or end, then sweep once — most interval problems fall to a single well-chosen sort plus a linear pass.

PLATE 16.01 Insert Interval [MEDIUM]

Given a sorted, non-overlapping list of intervals and a new interval, insert it merging as needed.

[View on LeetCode ↗](#)

APPROACH — Single pass: copy intervals before the overlap, merge the overlapping run, copy the rest

```
func insert(_ intervals: [[Int]], _ newInterval: [Int]) -> [[Int]] {  
    var result = [[Int]]()  
    var newInterval = newInterval  
    var i = 0  
    while i < intervals.count && intervals[i][1] < newInterval[0] {  
        result.append(intervals[i])  
        i += 1  
    }  
    while i < intervals.count && intervals[i][0] <= newInterval[1] {  
        newInterval[0] = min(newInterval[0], intervals[i][0])  
        newInterval[1] = max(newInterval[1], intervals[i][1])  
        i += 1  
    }  
    result.append(newInterval)  
    while i < intervals.count {  
        result.append(intervals[i])  
        i += 1  
    }  
    return result  
}
```

TIME $O(n)$

SPACE $O(n)$

PLATE 16.02 Merge Intervals [MEDIUM]

Given a collection of intervals, merge all overlapping ones.

[View on LeetCode ↗](#)

APPROACH — Sort by start, then sweep, extending or starting a new merged interval

```

func merge(_ intervals: [[Int]]) -> [[Int]] {
    let sorted = intervals.sorted { $0[0] < $1[0] }
    var result = [[Int]]()
    for interval in sorted {
        if var last = result.last, interval[0] <= last[1] {
            last[1] = max(last[1], interval[1])
            result[result.count - 1] = last
        } else {
            result.append(interval)
        }
    }
    return result
}

```

TIME $O(n \log n)$ SPACE $O(n)$

PLATE 16.03 Non-overlapping Intervals [MEDIUM]

Given a set of intervals, find the minimum number to remove so the rest don't overlap.

[View on LeetCode ↗](#)

APPROACH — Sort by end time, greedily keep the interval that finishes earliest

```

func eraseOverlapIntervals(_ intervals: [[Int]]) -> Int {
    let sorted = intervals.sorted { $0[1] < $1[1] }
    var removed = 0
    var end = Int.min
    for interval in sorted {
        if interval[0] >= end {
            end = interval[1]
        } else {
            removed += 1
        }
    }
    return removed
}

```

TIME $O(n \log n)$ SPACE $O(1)$

PLATE 16.04 Meeting Rooms [EASY]

Given meeting time intervals, determine if a single person could attend all of them.

[View on LeetCode ↗](#)

APPROACH – Sort by start time, check that no meeting starts before the previous one ends

```
func canAttendMeetings(_ intervals: [[Int]]) -> Bool {  
    let sorted = intervals.sorted { $0[0] < $1[0] }  
    for i in 1..  
sorted.count {  
        if sorted[i][0] < sorted[i-1][1] { return false }  
    }  
    return true  
}
```

TIME $O(n \log n)$ SPACE $O(1)$

PLATE 16.05 Meeting Rooms II [MEDIUM]

Given meeting time intervals, find the minimum number of conference rooms required.

[View on LeetCode ↗](#)

APPROACH – Sweep sorted start and end times separately, tracking concurrently active meetings

```
func minMeetingRooms(_ intervals: [[Int]]) -> Int {  
    let starts = intervals.map { $0[0] }.sorted()  
    let ends = intervals.map { $0[1] }.sorted()  
    var rooms = 0, maxRooms = 0  
    var s = 0, e = 0  
    while s < starts.count {  
        if starts[s] < ends[e] {  
            rooms += 1  
            s += 1  
        } else {  
            rooms -= 1  
            e += 1  
        }  
        maxRooms = max(maxRooms, rooms)  
    }  
    return maxRooms  
}
```

TIME $O(n \log n)$ SPACE $O(n)$

PLATE 16.06 Minimum Interval to Include Each Query [HARD]

Given intervals and queries, for each query find the size of the smallest interval that contains it.

[View on LeetCode ↗](#)

APPROACH – Sort both intervals and queries; sweep with a min-heap of active intervals ordered by size

```

func minInterval(_ intervals: [[Int]], _ queries: [Int]) -> [Int] {

    let sortedIntervals = intervals.sorted { $0[0] < $1[0] }

    let sortedQueries = queries.enumerated().sorted { $0.element < $1.element }

    var heap = Heap<(size: Int, end: Int)>(sort: { $0.size < $1.size })

    var result = [Int: Int]()

    var i = 0

    for (originalIndex, q) in sortedQueries {

        while i < sortedIntervals.count && sortedIntervals[i][0] <= q {

            let size = sortedIntervals[i][1] - sortedIntervals[i][0] + 1

            heap.push((size, sortedIntervals[i][1]))

            i += 1

        }

        while let top = heap.peek, top.end < q {

            _ = heap.pop()

        }

        result[originalIndex] = heap.peek?.size ?? -1

    }

    return queries.indices.map { result[$0]! }

}

```

TIME $O((n + q) \log n)$ SPACE $O(n + q)$

FAMILY 17

Math & Geometry

Problems that lean on number properties, matrix manipulation, or geometric traversal rather than one named algorithmic pattern.

PLATE 17.01 Rotate Image [MEDIUM]

Given an $n \times n$ matrix, rotate it 90 degrees clockwise in place.

[View on LeetCode ↗](#)

APPROACH – Transpose the matrix, then reverse each row

```
func rotate(_ matrix: inout [[Int]]) {  
    let n = matrix.count  
    for i in 0..  
n {  
        for j in (i+1)..  
n {  
            let temp = matrix[i][j]  
            matrix[i][j] = matrix[j][i]  
            matrix[j][i] = temp  
        }  
    }  
    for i in 0..  
n {  
        matrix[i].reverse()  
    }  
}
```

TIME $O(n^2)$ SPACE $O(1)$

PLATE 17.02 Spiral Matrix [MEDIUM]

Given a matrix, return all its elements in spiral order.

[View on LeetCode ↗](#)

APPROACH – Shrink four boundaries (top, bottom, left, right) inward as each side is traversed

```

func spiralOrder(_ matrix: [[Int]]) -> [Int] {
    var result = [Int]()
    var top = 0, bottom = matrix.count - 1
    var left = 0, right = matrix[0].count - 1
    while top <= bottom && left <= right {
        for c in left...right { result.append(matrix[top][c]) }
        top += 1
        if top > bottom { break }
        for r in top...bottom { result.append(matrix[r][right]) }
        right -= 1
        if left > right { break }
        for c in stride(from: right, through: left, by: -1)
        { result.append(matrix[bottom][c]) }
        bottom -= 1
        if top > bottom { break }
        for r in stride(from: bottom, through: top, by: -1) { result.append(matrix[r]
[left]) }
        left += 1
    }
    return result
}

```

TIME **O(rows · cols)** SPACE **O(1) extra**

PLATE 17.03 **Set Matrix Zeroes** [MEDIUM]

Given an $m \times n$ matrix, if an element is 0, set its entire row and column to 0, in place.

[View on LeetCode ↗](#)

APPROACH — Use the first row and column themselves as markers, tracked with two extra flags

```

func setZeroes(_ matrix: inout [[Int]]) {

    let rows = matrix.count, cols = matrix[0].count

    var firstRowZero = false, firstColZero = false

    for c in 0..

```

TIME $O(\text{rows} \cdot \text{cols})$ SPACE $O(1)$

PLATE 17.04 **Happy Number** [EASY]

A happy number eventually reaches 1 by repeatedly replacing it with the sum of squares of its digits. Determine if n is happy.

[View on LeetCode ↗](#)

APPROACH – Cycle detection with a Set; if a number repeats without hitting 1, it never will


```

func isHappy(_ n: Int) -> Bool {
    var seen = Set<Int>()

    var num = n

    func sumOfSquares(_ x: Int) -> Int {
        var x = x, sum = 0

        while x > 0 {
            let digit = x % 10
            sum += digit * digit
            x /= 10
        }

        return sum
    }

    while num != 1 && !seen.contains(num) {
        seen.insert(num)
        num = sumOfSquares(num)
    }

    return num == 1
}

```

TIME **$O(\log n)$** SPACE **$O(\log n)$**

PLATE 17.05 **Plus One** [EASY]

Given a large integer as an array of digits, add one to it and return the resulting digit array.

[View on LeetCode ↗](#)

APPROACH — Add from the rightmost digit, carrying only where needed

```

func plusOne(_ digits: [Int]) -> [Int] {
    var digits = digits

    for i in stride(from: digits.count - 1, through: 0, by: -1) {
        if digits[i] < 9 {
            digits[i] += 1
            return digits
        }

        digits[i] = 0
    }

    return [1] + digits
}

```

TIME **$O(n)$** SPACE **$O(1)$ extra**

PLATE 17.06 **Pow(x, n)** [MEDIUM]

Implement pow(x, n), computing x raised to the power n.

[View on LeetCode ↗](#)

APPROACH – Fast exponentiation by squaring, $O(\log n)$ time

```
func myPow(_ x: Double, _ n: Int) -> Double {  
    var base = x, exp = n  
  
    if exp < 0 { base = 1 / base; exp = -exp }  
  
    var result = 1.0  
  
    while exp > 0 {  
        if exp % 2 == 1 { result *= base }  
  
        base *= base  
  
        exp /= 2  
    }  
  
    return result  
}
```

TIME $O(\log n)$ SPACE $O(1)$

PLATE 17.07 **Multiply Strings** [MEDIUM]

Given two non-negative integers as strings, return their product, also as a string, without using built-in big-integer types.

[View on LeetCode ↗](#)

APPROACH – Grade-school long multiplication into a fixed-size digit buffer, propagating carries

```

func multiply(_ num1: String, _ num2: String) -> String {

    if num1 == "0" || num2 == "0" { return "0" }

    let a = Array(num1.reversed()).map { Int(String($0))! }
    let b = Array(num2.reversed()).map { Int(String($0))! }

    var result = [Int](repeating: 0, count: a.count + b.count)

    for i in 0..

```

TIME $O(m \cdot n)$ SPACE $O(m + n)$

PLATE 17.08 **Detect Squares** [MEDIUM]

Design a data structure that adds points and counts how many axis-aligned squares can be formed with a given point as a corner.

[View on LeetCode ↗](#)

APPROACH — Track point counts by (x, y) ; for each same- x point, check the two possible square completions

```

class DetectSquares {
    private var counts = [Int: [Int: Int]]() // x -> y -> count

    func add(_ point: [Int]) {
        counts[point[0], default: [:]][point[1], default: 0] += 1
    }

    func count(_ point: [Int]) -> Int {
        let (x, y) = (point[0], point[1])
        guard let yCounts = counts[x] else { return 0 }
        var total = 0
        for (otherY, yCount) in yCounts where otherY != y {
            let side = otherY - y
            for otherX in [x + side, x - side] {
                let count1 = counts[otherX]?[y] ?? 0
                let count2 = counts[otherX]?[otherY] ?? 0
                total += yCount * count1 * count2
            }
        }
        return total
    }
}

```

TIME **O(1) add, O(n) count** SPACE **O(n)**

FAMILY 18

Bit Manipulation

XOR, shifts, and masks solve problems in $O(1)$ extra space that would otherwise need a hash set.

PLATE 18.01 Single Number [EASY]

Given an array where every element appears twice except one, find that single one.

[View on LeetCode ↗](#)

APPROACH — XOR everything; identical pairs cancel to zero, leaving the singleton

```
func singleNumber(_ nums: [Int]) -> Int {  
    nums.reduce(0, ^)  
}
```

TIME $O(n)$ SPACE $O(1)$

PLATE 18.02 Number of 1 Bits [EASY]

Given an unsigned integer, return the number of '1' bits it has (Hamming weight).

[View on LeetCode ↗](#)

APPROACH — Repeatedly check the lowest bit and shift right

```
func hammingWeight(_ n: Int) -> Int {  
    var n = n, count = 0  
    while n != 0 {  
        count += n & 1  
        n >>= 1  
    }  
    return count  
}
```

TIME $O(32)$ SPACE $O(1)$

PLATE 18.03 Counting Bits [EASY]

Given n , return an array where $\text{ans}[i]$ is the number of 1 bits in the binary representation of i , for $0 \leq i \leq n$.

[View on LeetCode ↗](#)

APPROACH — $\text{dp}[i] = \text{dp}[i \gg 1] + (i \& 1)$ — reuse the answer for i with its last bit removed

```
func countBits(_ n: Int) -> [Int] {  
    var dp = [Int](repeating: 0, count: n + 1)  
    for i in 1...max(n, 1) where i <= n {  
        dp[i] = dp[i >> 1] + (i & 1)  
    }  
    return dp  
}
```

TIME $O(n)$ SPACE $O(n)$

PLATE 18.04 Reverse Bits [EASY]

Reverse the bits of a given 32-bit unsigned integer.

[View on LeetCode ↗](#)

APPROACH – Build the result bit by bit, shifting the input right and the result left

```
func reverseBits(_ n: UInt32) -> UInt32 {  
    var result: UInt32 = 0  
  
    var n = n  
  
    for _ in 0..  
32 {  
        result = (result << 1) | (n & 1)  
  
        n >>= 1  
    }  
  
    return result  
}
```

TIME **O(32)** SPACE **O(1)**

PLATE 18.05 Missing Number [EASY]

Given an array containing n distinct numbers from [0, n], find the one missing number.

[View on LeetCode ↗](#)

APPROACH – XOR every index and value together; everything present cancels, leaving the missing number

```
func missingNumber(_ nums: [Int]) -> Int {  
    var result = nums.count  
  
    for (i, n) in nums.enumerated() {  
        result ^= i ^ n  
    }  
  
    return result  
}
```

TIME **O(n)** SPACE **O(1)**

PLATE 18.06 Sum of Two Integers [MEDIUM]

Add two integers without using the + or - operators.

[View on LeetCode ↗](#)

APPROACH – XOR gives the sum without carrying, AND-then-shift gives the carry; repeat until no carry remains

```

func getSum(_ a: Int, _ b: Int) -> Int {

    var a = a, b = b

    while b != 0 {

        let carry = (a & b) << 1

        a = a ^ b

        b = carry

    }

    return a

}

```

TIME **O(1)** SPACE **O(1)**

PLATE 18.07 **Reverse Integer** [MEDIUM]

Given a 32-bit signed integer, reverse its digits. Return 0 if the reversed value overflows.

[View on LeetCode ↗](#)

APPROACH — Peel off digits with %10 and /10, rebuild in reverse, checking Int32 bounds each step

```

func reverse(_ x: Int) -> Int {

    var x = x

    var result = 0

    while x != 0 {

        let digit = x % 10

        x /= 10

        result = result * 10 + digit

        if result > Int32.max || result < Int32.min { return 0 }

    }

    return result

}

```

TIME **O(log n)** SPACE **O(1)**

